

Introduction to Databases

(INFR10080)

(Course Introduction)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Course Objectives

You will learn:

- What is a database/database system?
- How to query a database?
- How to design a database?
- How to update a database?
- How to use a database in your applications?

This course dose not cover:

- How to develop database systems* ⇐ check out [ADBS](#)
- non-relational databases ⇐ check out [MLS](#)

* But knowing how they work would help with using databases (advance topics)

2/29

Why you should take this course

SQL's enduring popularity (over half-century old)

- go-to/only language for accessing and operating databases
- popular language among professional developers for decades

\$\$\$...

Oracle

Dan Milmo *Global
technology editor*

Wed 10 Sep 2025 23.45 BST

 **Share**

Larry Ellison briefly overtakes Elon Musk as world's richest person

Oracle co-founder's shares rose by 40% in early trading, valuing his fortune at \$393bn, just ahead of Musk's \$384bn



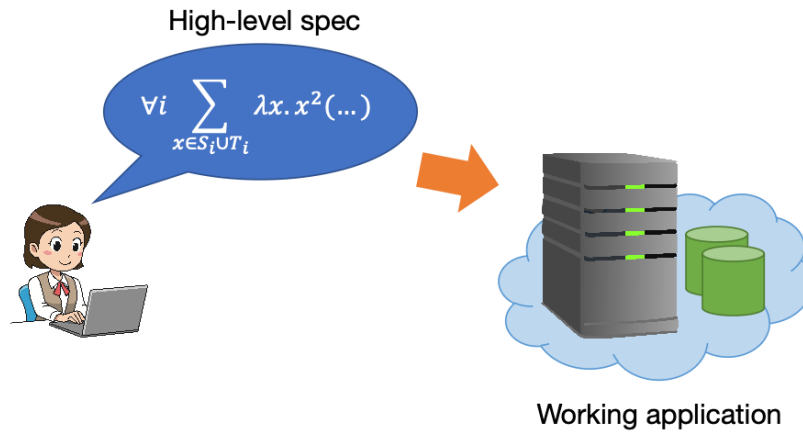
Source: theguardian.com/technology/2025/sep/10/larry-ellison-dislodges-elon-musk-as-worlds-richest-person

3/29

Why you should take this course

In many ways, SQL systems are the highest-level successful programming abstractions

Programming: The Dream



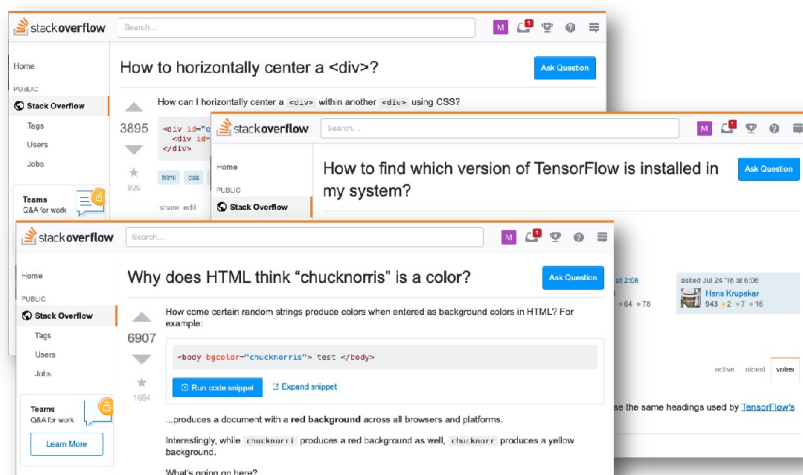
Source: CS 245: Principles of Data-Intensive Systems (Winter 2021), Stanford University <https://cs245.stanford.edu>

4/29

Why you should take this course

In many ways, SQL systems are the highest-level successful programming abstractions

Programming: The Reality



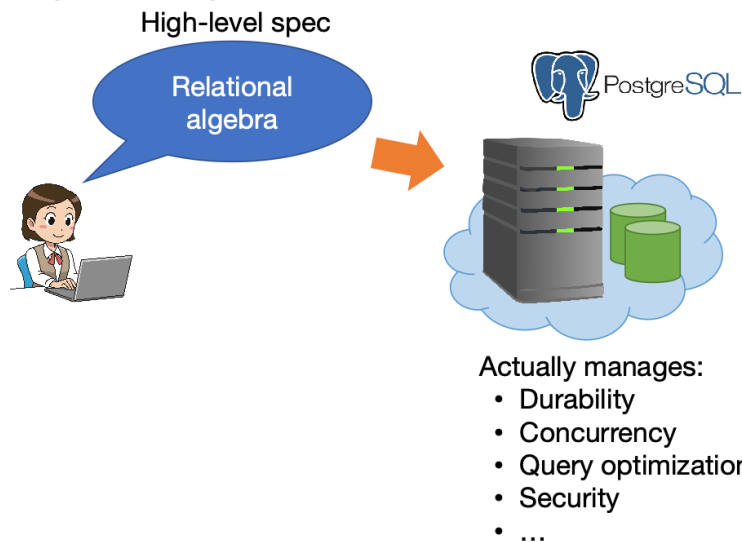
Source: CS 245: Principles of Data-Intensive Systems (Winter 2021), Stanford University <https://cs245.stanford.edu>

4/29

Why you should take this course

In many ways, SQL systems are the highest-level successful programming abstractions

Programming with Databases



Source: CS 245: Principles of Data-Intensive Systems (Winter 2021), Stanford University <https://cs245.stanford.edu>

4/29

Data

The **most important asset** of any enterprise

Must be *effectively*, *efficiently* and *reliably*

- collected and stored
- maintained and updated
- processed and analysed

to be *turned into meaningful information*

⇒ Enable and **support decision making**

5/29

What is a database?

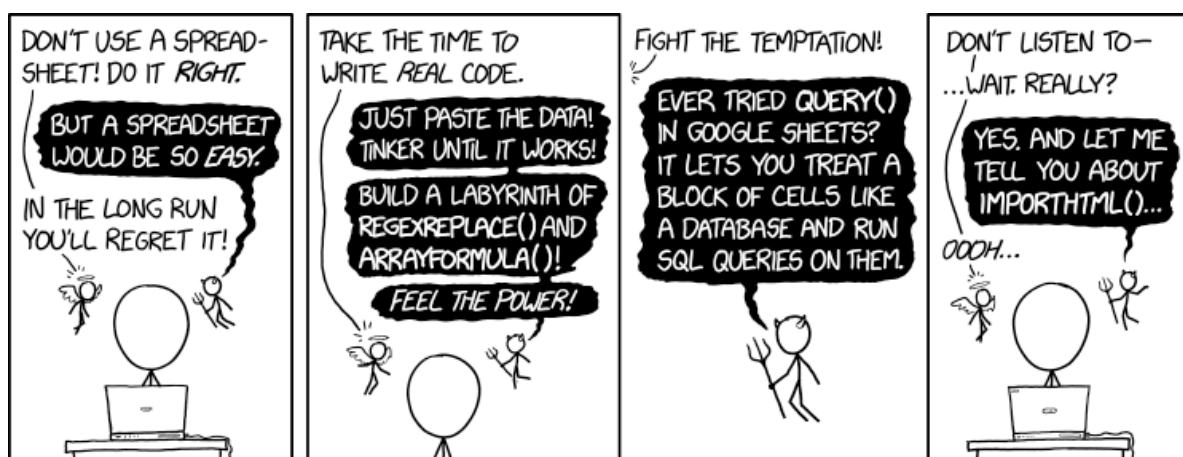
A collection of data items related to a specific enterprise, which is structured and organized so as to be more easily accessed, managed, and updated

Database Management System (DBMS)

- software package for creating and managing databases
- mediates interaction between end-users (incl. applications) and the database
- ensures that data is consistently organized and remains easily accessible

6/29

Why use a DBMS?



Source: <https://xkcd.com/2180>

7/29

Why use a DBMS?

- Uniform data administration
- Efficient access to resources
- Data independence
- Reduced application development time
- Data integrity and security
- Concurrent access
- Recovery from crashes

8/29

Different kinds of data(bases)

- A **data model** is a collection of concepts for describing data
- A **schema** is a description of a particular collection of data, using a given data model

Relational databases

⇐ main focus of this course

Data organised in tables (relations) with typed attributes

Document stores

Text documents structured using tags (or other markers)

Graph databases

Data organised in graph structures with nodes and edges

Vector databases

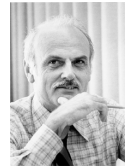
(Unstructured) data represented as embedding vectors

9/29

The relational model

First proposed by Edgar F. Codd in 1970

Simple idea: Organise data in **tables** (relations)



Schema

- Set of **table names**
- List of distinct (typed) **column names** for each table
- **Constraints** within a table or between tables

Instance

- Actual data (that is, the rows of the tables)
- Must satisfy typing and constraints

10/29

Example: relational database

Customer

CustID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

Account

Number	Branch	CustID	Balance
243576	Edinburgh	cust1	−120.00
250018	London	cust3	5621.73
745622	Manchester	cust2	1503.82

11/29

Query languages

Used to ask questions (**queries**) to a database

Procedural

Specify a **sequence of steps**
to obtain the expected result

Declarative

Specify **what** you want
not **how** to get it

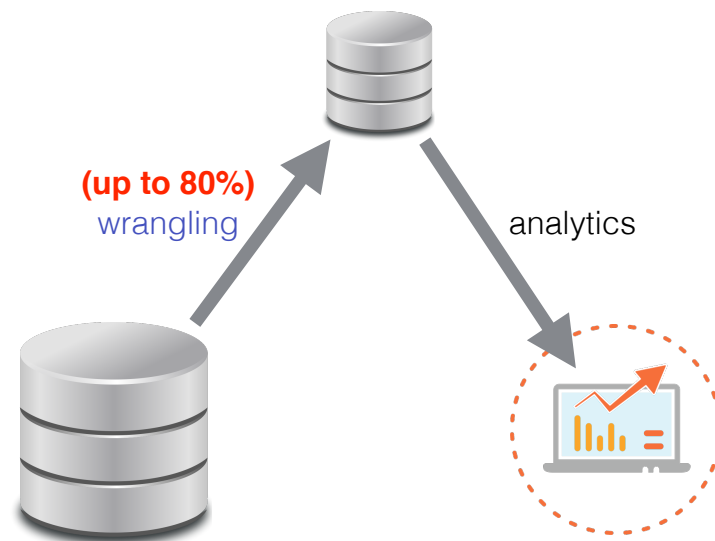
- Queries are typically asked in a declarative way
- DBMSs figure out internally how to translate a query into procedures that are suitable for getting the results

12/29

SQL

- Structured Query Language
- **Declarative** language for querying relational databases
- Implemented in all major (free and commercial) RDBMSs
- First **standardized** in 1986 (ANSI) and 1987 (ISO); several revisions afterwards (latest June 2023)
- Multi-billion-dollar business
- Most common tool used by **data scientists**

13/29



14/29

Studying SQL is not enough

DBMSs encompass many areas of Computer Science:

Operating systems, Algorithms and data structures,
Formal logic, (Programming) languages, Multimedia, ...

Goals of this course

- **Create and modify a relational database**
using standard software tools available on the market
- Compare strengths and weaknesses of different **database designs**
- Process and analyse data by means of **complex SQL statements**
- Formulate and manipulate queries
in both **declarative and procedural database languages**
- Reason about the correctness and consistency
of **concurrent database interactions** among multiple users

15/29

Syllabus

Core topics

- Query languages: **SQL**, relational algebra and calculus
- Database design: constraints and normal forms
- Scheduling and concurrency control: serializability, locking
- Database access from applications: embedded/dynamic SQL

Advanced topics (if time allows)

- Deductive databases: Datalog and recursive queries
- Incomplete data: missing values and certain answers
- Query evaluation and optimisation: join strategies, query plans
- Storage and indexing: B+ trees, static hashing

16/29

Prerequisites

- Some background in discrete mathematics
- Familiarity with **predicate logic** is a plus
(but this will be introduced during the course)
- Familiarity with **Unix command line** is a plus
(knowing the basics will make your life easier)
- No specific programming requirements
(we will see some very simple Python programs)

The course is overall self-contained

17/29

Course Staff

Instructor: Yang Cao

lecturers, office hours (in-person)

Course TA: Tianjian Yang

office hours (online via Collaborate)

more details later

18/29

Textbook (1)

Main text

Ramakrishnan, Gehrke:
Database Management Systems
McGraw-Hill, 3rd edition

- Available from the library
- **Not mandatory**
Lectures + materials posted on Learn are enough
(this is what the assessment is based on)

19/29

Textbook (2)

Further reading

Abiteboul, Vianu, Hull
Foundations of Databases
Addison-Wesley, 1995

- Mostly theoretical topics
- Out of print but freely available (for **personal use only**)
<http://webdam.inria.fr/Alice/>

20/29

Course website(s)

All **course materials** and **announcements** are on **Learn**

Class **discussions** will take place on **Piazza**

The link to the class is on Learn

All students enrolled in Learn are automatically signed up on Piazza

Rather than emailing questions, post them on Piazza

- You can post **privately** to instructors
- You can post **anonymously** to classmates

Most questions will be answered during office hours

21/29

Lectures

Weeks 1-11

Three hours per week

Thursdays 15:10-16:00

Lecture Theatre G.04 - Robson Building

Fridays 10:00-12:00

Usha Kasera Lecture Theatre - Old College

Week 11 is dedicated to **revision** (we typically solve a past exam)

22/29

Tutorials

There will be no formal tutorials required to attend.

Instead, tutorials are replaced by **Exercises** and **Office Hours**.

Exercises

Formative exercises: questions with detailed solutions Exercises will be made available in advance for self-study

Strongly advised to

1. check the lecture slides and exercise before each lecture
2. review the materials after each lecture
3. attempt the assigned exercises
4. ask questions on Piazza & during office hours

23/29

Office hours

Tailored 1-1 learning for your particular case. More effective than forced in-classroom tutorials.

Time

- Once a week, **from Week 2**, on **Thursdays** 13:45-14:45
- In person in my office (**Informatics Forum**, room **4.32A**)
- Online on **Blackboard Collaborate**:
<https://eu.bbcollab.com/guest/f8d181c853b540bfae434e490627462c>

During these sessions, we will **answer questions**:

- asked live (with priority to those who come in person)
- selected from those posted on Piazza

24/29

Coursework

Formative assignment (optional, but strongly advised)

- Requires writing queries in SQL and Relational Algebra
- Released in **Week 5**, due in **Week 6**
- Submission via Learn
- Feedback: individual automarked results + class discussions

Summative assignment (accounts for **20%** of the final mark)

- Similar to the formative assignment but **marked**
- Released in **Week 8**, due in **Week 9**
- Submission via Learn
- Feedback: individual automarked results

25/29

Exam

Accounts for **80%** of the final mark

Diets

December 2025 main examination, open to all students

August 2026 resit examination, still unclear who is allowed

Structure

- 6-8 questions, all compulsory for full marks
- For sample questions, search for
 - ▶ Database Systems (up until 2019), or
 - ▶ Introduction to Databases (from 2020)

at <https://exampapers.ed.ac.uk/>

26/29

Software: PostgreSQL

- Open-source, commercial-level relational DBMS
- Available for Windows, Mac, Linux (and more)
- Installed on all **DICE machines**
- Each enrolled student has their own **personal database** (hosted on the university's central PostgreSQL server)
- Instructions for using PostgreSQL on DICE are on Learn
- Used in the coursework
- More details later...

(We may also use other DBMSs (e.g., SQLite, duckdb) for in-classroom demos)

27/29

Software: REAL

- A cross-platform **interpreter for relational algebra**
- Written in Java, open-source
- Public Git repository hosted at
<https://git.ecdf.ed.ac.uk/pguaglia/real>
- This is an **educational tool** not meant for production
- Used in the coursework
- More details later...

28/29

Other stuff

Video recordings

- All lectures will be recorded
- The **recordings** will be available on **echo360** (link on Learn)

Lecture slides

- Handouts for all topics will be made available on Learn
- Keep an eye out for the **latest version**

In case of changes

before class: updated version number next to the link

after class: an announcement will also be sent out

29/29

Introduction to Databases

(INFR10080)

(Basic SQL)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

v25.1 Move ALTER TABLE to "Modifying Databases" section

The data model of SQL

Data is organised in **tables** (also called **relations**)
which are collections of **tuples** (also called **rows** or **records**)
which are **all of the same length**

Schema

- Set of **table names**
- List of **typed distinct column names**
(also called **attributes**) for each table
- **Constraints** within a table or between tables ⇐ not for now

Instance

- Actual data (that is, the rows of the table)
- Must satisfy typing and constraints

2/29

SQL

- Structured Query Language
- **Declarative** language for relational databases
- Implemented in all major (free and commercial) RDBMSs
- International Standard since 1987 (latest rev. June 2023)
- Consists of two sublanguages:
 - DDL** (Data Definition Language)
operations on the schema
 - DML** (Data Manipulation Language)
operations on the instance

3/29

Creating database

Creating tables

Basic syntax

```
CREATE TABLE table_name (  
    column_name1 column_type1,  
    column_name2 column_type2,  
    ⋮  
    column_namen column_typen  
) ;
```

Example

```
CREATE TABLE Customer (  
    custid varchar(10),  
    name varchar(20),  
    city varchar(30),  
    address varchar(30)  
) ;
```

Most common SQL data types

Strings

- `varchar(n)` – variable length, at most n characters

Numbers

- `smallint`
- `integer` or `int`
- `bigint`
- `numeric(p,s)` – arbitrary precision number
At most p total digits with s digits in the fractional part

Date & Time

- `date` – e.g., '2016-10-03'
- `time` – time of the day: e.g., '21:09'
- `timestamp`

5/29

Default values

Syntax

```
CREATE TABLE table_name (  
    column_name1 column_type1 [DEFAULT value1],  
    column_name2 column_type2 [DEFAULT value2],  
    ⋮  
    column_namen column_typen [DEFAULT valuen]  
);
```

Example

```
CREATE TABLE Account (  
    accnum varchar(12),  
    branch varchar(30),  
    custid varchar(10),  
    balance numeric(14,2) DEFAULT 0  
);
```

6/29

Populating tables

General syntax

```
INSERT INTO table_name VALUES (...), ..., (...);
```

Examples

```
INSERT INTO Account VALUES  
('243576', 'Edinburgh', 'cust1', -120);
```

```
INSERT INTO Customer VALUES  
('cust1', 'Renton', 'Edinburgh', '2 Wellington Pl'),  
('cust2', 'Watson', 'London', '221B Baker St'),  
('cust3', 'Holmes', 'London', '221B Baker St');
```

7/29

Populating tables with default values

Two possibilities:

1. Use the keyword **DEFAULT** in **INSERT**

Example

```
INSERT INTO Account VALUES  
('250018', 'London', 'cust3', DEFAULT);
```

2. List attributes explicitly (omitted ones will get the default)

Example

```
INSERT INTO Account (accnum, branch, custid) VALUES  
('250018', 'London', 'cust3');
```

Attributes without **DEFAULT** in **CREATE TABLE**
have default value **NULL** $\Leftarrow$ **more on this later**

8/29

Querying your tables

An extremely simple example

Customer			
ID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

List all attributes of all customers

```
SELECT *  
FROM Customer ;
```

* means “all attributes”

A very simple example

Customer

ID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

List name and address of all customers

```
SELECT Name, Address
FROM Customer ;
```

Output:

Name	Address
Renton	2 Wellington Pl
Watson	221B Baker St
Holmes	221B Baker St

10/29

A simple example

Customer

ID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

List name and address of customers living in Edinburgh

```
SELECT Name, Address
FROM Customer
WHERE City = 'Edinburgh' ;
```

Output:

Name	Address
Renton	2 Wellington Pl

11/29

The basic WHERE clause

term :=
| attribute
| value

comparison :=
| term op term, with op ∈ {=, <>, <, >, <=, >=}
| term IS NULL
| term IS NOT NULL

SELECT ...
FROM ...
WHERE *<condition>* ;

condition :=
| comparison
| condition AND condition
| condition OR condition
| NOT condition

12/29

Limiting Output

Customer			
ID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St
...	...	...	

List all attributes of all customers

SELECT *
FROM Customer ;

What if we have a very large table with too many rows?

LIMIT n

13/29

Ordering

ORDER BY $\langle \text{column}_1 \rangle$ [**DESC**], ..., $\langle \text{column}_n \rangle$ [**DESC**]

Sorts the output rows according to the values of column_1

If two rows have the same value for column_1 ,
they are sorted by the values of column_2 and so on ...

- Default ordering is **ascending** (can be specified with **ASC**)
- **Descending** ordering is specified by **DESC**

14/29

Ordering example (1)

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

```
SELECT      *  
FROM        Account  
ORDER BY    custid ASC, balance DESC ;
```

Number	Branch	CustID	Balance
111	London	1	1330.00
333	Edinburgh	1	450.00
222	London	2	1756.00

15/29

Ordering example (2)

Account

Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

```
SELECT *  
FROM Account  
ORDER BY custid DESC, balance ASC ;
```

Number	Branch	CustID	Balance
222	London	2	1756.00
333	Edinburgh	1	450.00
111	London	1	1330.00

Relating tables

More than one table in FROM

Table1	
A	B
1	2
3	4
1	3

Table2		
C	D	E
2	1	0
3	2	1

```
SELECT B, C
FROM Table1, Table2 ;
```

1. Each row of Table1 is **concatenated** with every row of Table2

A	B	C	D	E
1	2	2	1	0
1	2	3	2	1
3	4	2	1	0
3	4	3	2	1
1	3	2	1	0
1	3	3	2	1

17/29

More than one table in FROM

Table1	
A	B
1	2
3	4
1	3

Table2		
C	D	E
2	1	0
3	2	1

```
SELECT B, C
FROM Table1, Table2 ;
```

2. For each resulting row, the values of B and C are returned

B	C
2	2
4	2
3	2
2	3
4	3
3	3

17/29

Joining tables

Customer			Account		
ID	Name	City	AccNum	CustID	Balance
cust1	Renton	Edinburgh	123321	cust3	1330.00
cust2	Watson	London	243576	cust1	-120.00
cust3	Holmes	London			

List customers' names and their accounts' numbers

```
SELECT Name, AccNum
FROM Customer, Account
WHERE ID = CustID ;
```

Semantics: nested loop over the tables listed in FROM

Output:	Name	AccNum
	Renton	243576
	Holmes	123321

18/29

Basic queries in SQL

The basic pattern of SQL queries:

```
SELECT <comma-separated list of value expressions>
FROM <comma-separated list of tables>
WHERE <condition> ;
```

Idea

1. Loop over all rows of the tables listed in **FROM**
2. Take those that satisfy the **WHERE** condition
3. Compute and output the values listed in **SELECT**

The basic WHERE clause

term :=

- | attribute
- | value

comparison :=

- | term op term, with $op \in \{=, <>, <, >, <=, >=\}$
- | term IS NULL
- | term IS NOT NULL

condition :=

- | comparison
- | condition AND condition
- | condition OR condition
- | NOT condition

20/29

WHERE conditions in queries

- **filter** data within a table

```
SELECT Name, Address
FROM   Customer
WHERE  City = 'Edinburgh' ;
```

- **join** data from different tables

```
SELECT Name, AccNum
FROM   Customer, Account
WHERE  ID = CustID ;
```

Filtering and join together

```
SELECT Name, Address, AccNum
FROM   Customer, Account
WHERE  ID = CustID AND City = 'Edinburgh' ;
```

21/29

Explicit join syntax

```
table1 JOIN table2 ON ⟨condition⟩1  
⋮  
JOIN tablen ON ⟨condition⟩n-1
```

Logically separate join conditions from filters

```
SELECT Name, Balance  
FROM Customer, Account  
WHERE ID = CustID AND Balance < 0 ;
```

```
SELECT Name, Balance  
FROM Customer JOIN Account ON ID=CustID  
WHERE Balance < 0 ;
```

22/29

Qualification of attributes

Customer			Account		
CustID	Name	City	AccNum	CustID	Balance
cust1	Renton	Edinburgh	123321	cust3	1330.00
cust2	Watson	London	243576	cust1	-120.00
cust3	Holmes	London			

List the name of customers whose account is overdrawn

```
SELECT Customer.Name  
FROM Customer, Account  
WHERE Account.CustID = Customer.CustID  
AND Account.Balance < 0 ;
```

We need to specify the relations attributes are coming from

What is the output of this query?

23/29

Range variables

Assign new names to tables in FROM

```
SELECT Customer.Name, Account.Balance
FROM    Customer, Account
WHERE   Account.CustID = Customer.CustID
        AND Account.Balance < 0 ;
```

```
SELECT C.Name, A.Balance
FROM    Customer C, Account AS A
WHERE   A.CustID = C.CustID
        AND A.Balance < 0 ;
```

```
SELECT C.Name, A.Balance
FROM    Customer C JOIN Account A ON C.CustID=A.CustID
WHERE   A.Balance < 0 ;
```

24/29

Renaming attributes

```
SELECT C.Name CustName, A.Balance AS AccBal
FROM    Customer C, Account A
WHERE   A.CustID = C.CustID
        AND A.Balance < 0 ;
```

This does not work:

```
SELECT C.Name CustName, A.Balance AS AccBal
FROM    Customer C, Account A
WHERE   A.CustID = C.CustID
        AND AccBal < 0 ;
```

25/29

Modifying databases

Changing the definition of a table

```
ALTER TABLE name
    RENAME TO new_name ;
    RENAME column TO new_column ;
    ADD column type ;
    DROP column ;
    ALTER column
        TYPE type ;
        SET DEFAULT value ;
        DROP DEFAULT;
```

Destroying tables

TRUNCATE TABLE <i>name</i> ;	delete all rows from the table
DROP TABLE <i>name</i> ;	completely remove table from schema

Many other changes are possible ...

Database modification: Deletion

General form

```
DELETE FROM table_name  
WHERE  $\langle condition \rangle$  ;
```

All rows in *table_name* satisfying $\langle condition \rangle$ are deleted

Example

Remove accounts with zero balance and unknown owner

```
DELETE FROM Account  
WHERE Balance = 0 AND CustID IS NULL ;
```

27/29

Database modification: Replacement

General form

```
UPDATE table_name  
SET  $\langle assignments \rangle$   
WHERE  $\langle condition \rangle$  ;
```

Replace the values of some attributes (using $\langle assignments \rangle$)
in each row of *table_name* that satisfies $\langle condition \rangle$

Examples

Set a new balance on account 745622

```
UPDATE Account  
SET balance = 1503.82  
WHERE accnum = '745622' ;
```

Accounts in London with positive balance get a 0.2% bonus

```
UPDATE Account  
SET balance = balance + 0.002 * balance  
WHERE branch = 'London' AND balance > 0 ;
```

28/29

Concluding remarks

- SQL is case-insensitive (for keywords and table/column names)
but **string constants are case-sensitive**: 'abc' $\neq$ 'aBc'
- (SQL) queries are read-only
they do **not modify** the **schema**
nor the **instance** of the database
- Always use range variables (aliases for tables)
and fully qualify references to attributes
 - $\Rightarrow$ improves **readability** of queries
 - $\Rightarrow$ more **robust** against schema changes

Introduction to Databases

(INFR10080)

(Relational Algebra)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Data model

- A **relation** is a **set** of **records**
over the same **set** of (distinct) attribute names
- A **record** is a **total function** from attribute names to values

Schema

- Set of **relation names**
- Set of **distinct attributes** for each table
Note that columns are not ordered

Instance

- Actual data (that is, the records in each relation)

2/21

Relational algebra

Procedural query language

A relational algebra expression

- takes as input one or more relations
- applies a **sequence of operations**
- returns a relation as output

Operations:

Projection (π)

Selection (σ)

Product ($\times$)

Renaming (ρ)

Union ($\cup$)

Intersection ($\cap$)

Difference ($-$)

The application of each operation results in a new (virtual) relation that can be used as input to other operations

3/21

Projection

- **Vertical operation**: choose some of the columns
- **Syntax**: $\pi_{\text{set of attributes}}(\text{relation})$
- $\pi_{A_1, \dots, A_n}(R)$ takes only the values of attributes $A_1, \dots, A_n$ for each tuple in R

Customer

CustID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

$\pi_{\text{Name, City}}(\text{Customer})$

Name	City
Renton	Edinburgh
Watson	London
Holmes	London

4/21

Selection

- **Horizontal operation**: choose rows satisfying some condition
- **Syntax**: $\sigma_{\text{condition}}(\text{relation})$
- $\sigma_{\theta}(R)$ takes only the tuples in R for which θ is satisfied

term := attribute | constant

θ := **term** **op** **term** with **op** $\in \{=, \neq, >, <, \geq, \leq\}$
| $\theta \wedge \theta$ | $\theta \vee \theta$ | $\neg \theta$

5/21

Example of selection

Customer

CustID	Name	City	Age
cust1	Renton	Edinburgh	24
cust2	Watson	London	32
cust3	Holmes	London	35

$\sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer})$

CustID	Name	City	Age
cust2	Watson	London	32

6/21

Efficiency (1)

Consecutive selections can be combined into a single one:

$$\sigma_{\theta_1}(\sigma_{\theta_2}(R)) \equiv \sigma_{\theta_1 \wedge \theta_2}(R)$$

↳ is equivalent to: two queries Q_1 and Q_2 are **equivalent** if Q_1 returns the same answers as Q_2 on every database

Example

$$Q_1 = \sigma_{\text{City} \neq \text{'Edinburgh'}}(\sigma_{\text{Age} < 33}(\text{Customer}))$$

$$Q_2 = \sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer})$$

$Q_1 \equiv Q_2$ but Q_2 **faster** than Q_1 in general

7/21

Efficiency (2)

Projection can be pulled in front of selection

$$\sigma_{\theta}(\pi_{\alpha}(R)) \equiv \pi_{\alpha}(\sigma_{\theta}(R))$$

only if all attributes mentioned in θ appear in α

Example

$$Q_1 = \pi_{\text{Name, City, Age}}(\sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer}))$$

$$Q_2 = \sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\pi_{\text{Name, City, Age}}(\text{Customer}))$$

Question: Which one is more efficient?

8/21

Cartesian product

$R \times S$ **concatenates** each tuple of R with all the tuples of S

Note: the relations must have **disjoint** sets of attributes

Example

R	A	B	$\times$	S	C	D	=	$R \times S$	A	B	C	D
	1	2			1	a			1	2	1	a
	3	4			2	b			1	2	2	b
					3	c			1	2	3	c
									3	4	1	a
									3	4	2	b
									3	4	3	c

Expensive operation:

- $\text{card}(R \times S) = \text{card}(R) \times \text{card}(S)$
- $\text{arity}(R \times S) = \text{arity}(R) + \text{arity}(S)$

9/21

Joining relations

Combining Cartesian product and selection

Customer: ID, Name, City, Address

Account: Number, Branch, CustID, Balance

We can join customers with the accounts they own as follows

$$\sigma_{ID=CustID}(Customer \times Account)$$

10/21

Renaming

Gives new names to the attribute of a relation

Syntax: $\rho_{\text{replacement}_1, \dots, \text{replacement}_k}(\text{relation})$

where each **replacement** has the form **old** $\rightarrow$ **new**

Requirements

Let X is the set of attributes of the input relation, then

- The l.h.s. of each replacement belongs to X
- No two replacements have the same l.h.s.
- Composing the identity on X with all replacements results in an injective function

11/21

Examples of renaming

$$\bullet \rho_{B \rightarrow E} \left(\overbrace{\begin{array}{c|ccc} & \mathbf{A} & \mathbf{B} & \mathbf{C} \\ \hline a & b & c \\ 1 & 2 & 3 \end{array}}^R \right) = \begin{array}{c|ccc} & \mathbf{A} & \mathbf{E} & \mathbf{C} \\ \hline a & b & c \\ 1 & 2 & 3 \end{array}$$

- $\rho_{D \rightarrow E}(R)$ and $\rho_{B \rightarrow A}(R)$ are **illegal** (why?)

- $\rho_{A \rightarrow B, B \rightarrow A}(R)$ is **legal** (what does it do?)

- $\rho_{A \rightarrow B, B \rightarrow A, A \rightarrow E}(R)$ is also **illegal** (why?)

12/21

Natural join

Joins two tables for equality on their **common attributes**

Example

Customer: **CustID**, Name, City, Address

Account: Number, Branch, **CustID**, Balance

$\text{Customer} \bowtie \text{Account} \equiv$

$$\pi_{X \cup Y}(\sigma_{\text{CustID}=\text{CustID}'}(\text{Customer} \times \rho_{\text{CustID} \rightarrow \text{CustID}'}(\text{Account})))$$

where $X = \{ \text{all attributes of Customer} \}$

$Y = \{ \text{all attributes of Account} \}$

14/21

From SQL to relational algebra

SELECT $\mapsto$ projection π

FROM $\mapsto$ Cartesian product $\times$

WHERE $\mapsto$ selection σ

SELECT $A_1, \dots, A_m$
FROM $T_1, \dots, T_n$ $\mapsto \pi_{A_1, \dots, A_m}(\sigma_{\langle \text{condition} \rangle}(T_1 \times \dots \times T_n))$
WHERE $\langle \text{condition} \rangle$

Common attributes in $T_1, \dots, T_n$ must be renamed

15/21

Set operations

Union

R	A	B	$\cup$	S	A	B	=	R $\cup$ S	A	B
	a1	b1			a1	b1			a1	b1
	a2	b2			a3	b3			a2	b2
									a3	b3

Intersection

R	A	B	$\cap$	S	A	B	=	R $\cap$ S	A	B
	a1	b1			a1	b1			a1	b1
	a2	b2			a3	b3				

Difference

R	A	B	$-$	S	A	B	=	R $-$ S	A	B
	a1	b1			a1	b1			a2	b2
	a2	b2			a3	b3				

The relations must have the same set of attributes

16/21

Union and renaming

R	Father	Child	S	Mother	Child
	George	Elizabeth		Elizabeth	Charles
	Philip	Charles		Elizabeth	Andrew
	Charles	William			

We want to find the relation **parent-child**

$\rho_{\text{Father} \rightarrow \text{Parent}}(R) \cup \rho_{\text{Mother} \rightarrow \text{Parent}}(S)$	=	Parent	Child
		George	Elizabeth
		Philip	Charles
		Charles	William
		Elizabeth	Charles
		Elizabeth	Andrew

17/21

Full relational algebra

Primitive operations: $\pi, \sigma, \times, \rho, \cup, -$

Removing any of these results in a **loss of expressive power**

Derived operations

$\bowtie$ can be expressed in terms of $\pi, \sigma, \times, \rho$

$\cap$ can be expressed in terms difference:

$$R \cap S \equiv R - (R - S)$$

18/21

Other derived operations

Theta-join	$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$
Equijoin	$\bowtie_{\theta}$ where θ is a conjunction of equalities
Semijoin	$R \ltimes_{\theta} S \equiv \pi_X(R \bowtie_{\theta} S)$ where X is the set of attributes of R
Antijoin	$R \bar{\ltimes}_{\theta} S \equiv R - (R \ltimes_{\theta} S)$

Why use these operations?

- to write things more succinctly
- they can be optimized independently

19/21

Division

R over set of attributes X

S over set of attributes $Y \subset X$

Let $Z = X - Y$

$$\begin{aligned} R \div S &= \{ r \in \pi_Z(R) \mid \text{for every } s \in S, rs \in R \} \\ &= \{ r \in \pi_Z(R) \mid \{r\} \times S \subseteq R \} \\ &= \pi_Z(R) - \pi_Z(\pi_Z(R) \times S - R) \end{aligned}$$

20/21

Division: Example

Exams		DPT
Student	Course	Course
John	Databases	Databases
John	Networks	Programming
Mary	Programming	
Mary	Math	
Mary	Databases	

$$\text{Exams} \div \text{DPT} = \begin{array}{|c|} \hline \text{Student} \\ \hline \text{Mary} \\ \hline \end{array}$$

$$= \pi_{\text{Student}}(\text{Exams}) - \pi_{\text{Student}}(\pi_{\text{Student}}(\text{Exams}) \times \text{DPT} - \text{Exams})$$

Introduction to Databases

(INFR10080)

(Propositional Logic)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Building blocks

Propositions

Atomic statements that cannot be further decomposed

For example:

- “It is raining”
- “The cat is on the table”
- “The sky is blue”

Usually denoted with uppercase letters: $P, Q, \dots$

Also called **propositional variables**

Logical connectives

- Conjunction: $\wedge$ (**and**)
- Negation: $\neg$ (**not**)

2/23

Syntax

Let **Prop** be a countable set of propositional variables

The **language** $\mathcal{L}$ of propositional logic over **Prop** is defined inductively as follows:

1. Every $P \in \mathbf{Prop}$ is in $\mathcal{L}$ (**atomic formulae** or **atoms**)
2. If ϕ and ψ are in $\mathcal{L}$, then $\phi \wedge \psi$ is also in $\mathcal{L}$
3. If ϕ is in $\mathcal{L}$, then also $\neg\phi$ is in $\mathcal{L}$
4. Nothing else is in $\mathcal{L}$

In other words, $\mathcal{L}$ is generated by the following grammar:

$$\phi := P \mid \phi \wedge \phi \mid \neg\phi \quad (\text{with } P \in \mathbf{Prop})$$

3/23

Semantics

Intuition

- Atomic statements can be either **true** (t) or **false** (f)
- The truth value of a formula is determined by the truth values of its atoms

Formally

A **truth-value assignment** is a function $\alpha: \mathbf{Prop} \rightarrow \{\mathbf{t}, \mathbf{f}\}$

Then, α **satisfies** a formula ϕ (written $\alpha \models \phi$) inductively as follows:

$$\begin{array}{lll} \alpha \models P & \iff & \alpha(P) = \mathbf{t} \\ \alpha \models \neg\phi & \iff & \alpha \not\models \phi \\ \alpha \models \phi \wedge \psi & \iff & \alpha \models \phi \text{ and } \alpha \models \psi \end{array}$$

4/23

Truth tables

Reflect the semantics of the connectives

$\wedge$	t	f
t	t	f
f	f	f

	$\neg$
t	f
f	t

Given α and ϕ , allow us to determine whether $\alpha \models \phi$:

1. Replace each propositional variable P in ϕ by $\alpha(P)$
2. Propagate **t** and **f** according to the truth tables

5/23

Example

Given the formula

$$\phi = \neg(\neg A \wedge B) \wedge \neg(C \wedge \neg D)$$

and the assignment

$$\alpha = \{ A \mapsto \mathbf{t}, \quad B \mapsto \mathbf{f}, \quad C \mapsto \mathbf{f}, \quad D \mapsto \mathbf{t} \}$$

does α satisfy ϕ ?

Yes

Given α and ϕ , checking whether $\alpha \models \phi$
can be done in **polynomial time** in the size of ϕ

6/23

Satisfiability and Validity

A formula ϕ is

satisfiable if there is some α that satisfies ϕ

unsatisfiable if ϕ is not satisfiable

falsifiable if there is some α that does not satisfy ϕ

valid if every α satisfies ϕ
(in this case ϕ is called a **tautology**)

Consequences

- ϕ is a tautology iff $\neg\phi$ is unsatisfiable
- ϕ is unsatisfiable iff $\neg\phi$ is a tautology

7/23

Equivalence

Two formulas are **logically equivalent** (written $\phi \equiv \psi$) if for all α

$$\alpha \models \phi \iff \alpha \models \psi$$

(i.e., there is no assignment that satisfies one formula but not the other)

Intuition: Equivalent formulae have the **same meaning** even though they may **differ syntactically** (they say the same thing in different ways)

8/23

Propositional logic: Extended language

Syntax

New connectives:

- Disjunction $\vee$ (**or**)
- Implication $\rightarrow$ (**if-then**)
- Double implication $\leftrightarrow$ (**if and only if**)

New grammar:

$$\phi := P \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi \mid \phi \leftrightarrow \phi$$

Semantics

$\alpha \models \phi \vee \psi$	iff	$\alpha \models \phi$ or $\alpha \models \psi$
$\alpha \models \phi \rightarrow \psi$	iff	if $\alpha \models \phi$, then $\alpha \models \psi$
$\alpha \models \phi \leftrightarrow \psi$	iff	($\alpha \models \phi$ if and only if $\alpha \models \psi$)

9/23

Expressive power

Every formula in the extended language
can be **equivalently expressed** using only $\wedge$ and $\neg$

- $\phi \vee \psi \equiv \neg(\neg\phi \wedge \neg\psi)$
- $\phi \rightarrow \psi \equiv \neg\phi \vee \psi$
- $\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$

The new connectives do not add **expressive power** to the language

- they are just **syntactic sugar**
- but useful to write formulae more succinctly

10/23

Truth tables of the new connectives

Disjunction		
$\vee$	t	f
t	t	t
f	t	f

Implication		
$\rightarrow$	t	f
t	t	f
f	t	t

Double Implication		
$\leftrightarrow$	t	f
t	t	f
f	f	t

All of the above can be derived from the truth tables for $\wedge$ and $\neg$

11/23

Equivalences

Commutativity

$$\phi \vee \psi \equiv \psi \vee \phi$$

$$\phi \wedge \psi \equiv \psi \wedge \phi$$

Associativity

$$(\phi \vee \psi) \vee \chi \equiv \phi \vee (\psi \vee \chi)$$

$$(\phi \wedge \psi) \wedge \chi \equiv \phi \wedge (\psi \wedge \chi)$$

Distributivity

$$\phi \wedge (\psi \vee \chi) \equiv (\phi \wedge \psi) \vee (\phi \wedge \chi)$$

$$\phi \vee (\psi \wedge \chi) \equiv (\phi \vee \psi) \wedge (\phi \vee \chi)$$

Idempotence

$$\phi \vee \phi \equiv \phi$$

$$\phi \wedge \phi \equiv \phi$$

Absorption

$$\phi \vee (\phi \wedge \psi) \equiv \phi$$

$$\phi \wedge (\phi \vee \psi) \equiv \phi$$

12/23

Equivalences (continued)

Double Negation

$$\neg\neg\phi \equiv \phi$$

De Morgan

$$\neg(\phi \vee \psi) \equiv \neg\phi \wedge \neg\psi$$

$$\neg(\phi \wedge \psi) \equiv \neg\phi \vee \neg\psi$$

Implication

$$\phi \rightarrow \psi \equiv \neg\phi \vee \psi$$

13/23

Entailment

We extend the **satisfaction relationship** $\models$ to **sets Σ of formulae**:

$$\alpha \models \Sigma \iff \alpha \models \phi \text{ for all } \phi \in \Sigma \iff \alpha \models \bigwedge_{\phi \in \Sigma} \phi$$

Then, we say that Σ **entails** a formula ϕ if for all α

$$\alpha \models \phi \text{ whenever } \alpha \models \Sigma$$

(i.e., every assignment that satisfies Σ also satisfies ϕ)

14/23

Properties of Entailment

Deduction Theorem

$$\Sigma \cup \{\phi\} \models \psi \text{ iff } \Sigma \models \phi \rightarrow \psi$$

Contraposition Theorem

$$\Sigma \cup \{\phi\} \models \neg\psi \text{ iff } \Sigma \cup \{\psi\} \models \neg\phi$$

Contradiction Theorem

$$\Sigma \cup \{\phi\} \text{ is unsatisfiable iff } \Sigma \models \neg\phi$$

15/23

Decision problems

A **decision problem**

- takes (one or more) inputs
- answers a Yes / No question

Example

SAT: Given a propositional formula ϕ , is ϕ satisfiable?
(i.e., can we find an assignment α such that $\alpha \models \phi$?)

A **decision procedure** is an algorithm that

- always **terminates**
- **solves** a decision problem

A decision problem is **decidable** if there is decision procedure for it

16/23

Solving SAT

Satisfiability in propositional logic is a **decidable** problem

Naive algorithm

1. Enumerate all possible assignments
(there are 2^n where n is the number of atoms in the formula)
2. For each assignment check whether the formula is satisfied:
 - 2.1 if it is, stop and answer YES
 - 2.2 otherwise, continue to next assignment
3. If there are no more assignment, stop and answer NO

SAT is an **NP-complete** problem:

- a (candidate) solution can be verified in polynomial time
- no known efficient (polynomial) way to locate a solution
(in the worst case, it requires **exponential** time)

17/23

Reduction to satisfiability

Validity, equivalence, and entailment

can all be reduced to checking satisfiability:

- ϕ is valid iff $\neg\phi$ is not satisfiable
- $\phi \models \psi$ iff $\phi \rightarrow \psi$ is valid (deduction theorem)
- $\phi \equiv \psi$ iff $\phi \leftrightarrow \psi$ is valid

A decision procedure for satisfiability is all we need

Validity, equivalence and entailment are decidable problems
(in particular, **coNP-complete**) in propositional logic

18/23

Some additional terminology

Atom atomic formula

Literal atom (**positive literal**)
or its negation (**negative literal**)

Clause disjunction of literals

Term conjunction of literals

19/23

Normal forms

Formulae expressed in a standard syntactic form

Conjunctive Normal Form (CNF)

Conjunction of clauses: $\bigwedge_{i=1}^n (\bigvee_{j=1}^m L_{i,j})$

Example: $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

Disjunctive Normal Form (DNF)

Disjunction of terms: $\bigvee_{i=1}^n (\bigwedge_{j=1}^m L_{i,j})$

Example: $(A \wedge B) \vee (A \wedge \neg C) \vee (A \wedge \neg D) \vee (\neg B \wedge \neg C) \vee (\neg B \wedge \neg D)$

Negation Normal Form (NNF)

Only $\wedge$, $\vee$, $\neg$ and negation can appear only in front of atoms
(in particular, every formula in CNF or DNF is also in NNF)

Example: $(A \wedge (B \vee \neg C \vee \neg D)) \vee (\neg B \wedge (B \vee \neg C \vee \neg D))$

20/23

Converting to NNF, CNF and DNF

For every formula
there exist **equivalent** formulae in **CNF**, **DNF** and **NNF**

To convert into NNF

1. Replace $\{\rightarrow, \leftrightarrow\}$ by $\{\wedge, \vee, \neg\}$
2. Apply De Morgan's laws

To convert into CNF or DNF

1. Convert into NNF
2. Apply the distributivity laws

21/23

Why normal forms?

Some approaches to inference use syntactic operations on formulae, often expressed in standardized forms

CNF tells us something as to whether a formula is **valid**:

- If all clauses contain complementary literals, then the formula is a tautology
- Otherwise, the formula is falsifiable

DNF tells us something as to whether a formula is **satisfiable**:

- If all terms contain complementary literals, then the formula is unsatisfiable
- Otherwise, the formula is satisfiable

22/23

DNF and Satisfiability

Whether a formula in DNF is satisfiable can be decided in **linear time** in the size of the formula

Wait! Didn't we say that SAT is NP-complete? **Yes**

So what's the catch with DNF?

The transformation into DNF is **expensive** (in time/space)

- The size of the formula may **blow-up exponentially!**

23/23

Introduction to Databases

(INFR10080)

(Predicate Logic)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Logic in general

Logics are formal languages for

- representing **what we know** about the world
- **reasoning** about this knowledge (draw conclusions from it)

Two components:

Syntax defines the sentences in the language

Semantics defines the **meaning** of the sentences

Used in many areas of Computer Science:

- Artificial Intelligence
- Semantic Web
- Software & Hardware verification
- **Databases**
- ... many many others

2/22

Motivation for Predicate Logic

Atomic formulas of propositional logic are **too atomic**

- statements that may be true or false
- but have **no internal structure**

First-order (or **predicate**) **logic** (FOL) overcomes this limitation

- atomic formulas are statements about **relationships between objects**

3/22

Predicates and constants

Consider the statements:

- *Mary is happy*
- *John is rich*
- *Mary and John are siblings*

In propositional logic these are just atomic propositions:

- mary-is-happy
- john-is-rich
- mary-and-john-are-siblings

In first-order logic atomic statements use **predicates**, with **constants** as arguments:

- **Happy**(**Mary**)
- **Rich**(**John**)
- **Sibling**(**Mary**, **John**)

4/22

Variables and quantifiers

Consider the statements:

- *Someone is happy*
- *Being rich does not make one happy*

FOL predicates may have **variables** as arguments, whose value may be bound by **quantifiers**:

- $\exists x \text{ Happy}(x)$
- $\neg \forall x (\text{Rich}(x) \rightarrow \text{Happy}(x))$

5/22

Syntax of FOL: terms

Countably infinite supply of

variables : $x, y, z, \dots$

constants : $a, b, c, \dots$

predicates : $P, Q, R, \dots$ (with associated **arities**)

Term	$t := x$	variable
	$ a$	constant

6/22

Syntax of FOL: formulas

Formula	$\phi := P(t_1, \dots, t_n)$	atomic formula
	$ \neg \phi$	negation
	$ \phi \wedge \phi$	conjunction
	$ \phi \vee \phi$	disjunction
	$ \phi \rightarrow \phi$	implication
	$ \forall x \phi$	universal quantification (if x occurs free in ϕ)
	$ \exists x \phi$	existential quantification (if x occurs free in ϕ)

7/22

Quantifiers and free variables

Variables that are **not in the scope of any quantifier**

A variable that is not free is **bound**

Example: $\forall x (R(y, z) \wedge \exists y (\neg P(y, x) \vee R(y, z)))$

Variables in **blue** are free, the others are bound

We assume **quantifiers bind till the end** of the formula:

Example: the formula above can be written as

$$\forall x R(y, z) \wedge \exists y \neg P(y, x) \vee R(y, z)$$

Notation

We write $\exists x_1 \exists x_2 \cdots \exists x_n \phi$ as $\exists x_1, \dots, x_n \phi$
and $\forall x_1 \forall x_2 \cdots \forall x_n \phi$ as $\forall x_1, \dots, x_n \phi$

FOL interpretations

A formula may be true (or false) w.r.t. a given **interpretation** consisting of

- a **semantic function** $\mathcal{I}$ mapping each **predicate symbol** to a **relation** (over constants) of appropriate arity

Example: If Person is a binary predicate,
 $\text{Person}^{\mathcal{I}}$ could be $\{(Mary, 24), (John, 32), \dots\}$

- a **variable assignment** ν mapping each **variable** to a **constant**

Example: $\nu = \{x \mapsto 29, y \mapsto John, \dots\}$

Notation $\nu[x/a]$ is the same as ν except that $x \mapsto a$

Example: For ν above, $\nu[y/31] = \{x \mapsto 29, y \mapsto 31, \dots\}$

10/22

Semantics of FOL

We extend ν to be the **identity over constants**

(so that we can apply ν to all **terms**)

$\mathcal{I}, \nu \models \phi$ means the interpretation $(\mathcal{I}, \nu)$ **satisfies** formula ϕ

$\mathcal{I}, \nu \models P(t_1, \dots, t_n)$	$\iff$	$(\nu(t_1), \dots, \nu(t_n)) \in P^{\mathcal{I}}$
$\mathcal{I}, \nu \models \neg \phi$	$\iff$	$\mathcal{I}, \nu \not\models \phi$
$\mathcal{I}, \nu \models \phi \wedge \psi$	$\iff$	$\mathcal{I}, \nu \models \phi$ and $\mathcal{I}, \nu \models \psi$
$\mathcal{I}, \nu \models \phi \vee \psi$	$\iff$	$\mathcal{I}, \nu \models \phi$ or $\mathcal{I}, \nu \models \psi$
$\mathcal{I}, \nu \models \phi \rightarrow \psi$	$\iff$	if $\mathcal{I}, \nu \models \phi$ then $\mathcal{I}, \nu \models \psi$
$\mathcal{I}, \nu \models \forall x \phi$	$\iff$	for every constant a : $\mathcal{I}, \nu[x/a] \models \phi$
$\mathcal{I}, \nu \models \exists x \phi$	$\iff$	there is a constant a s.t. $\mathcal{I}, \nu[x/a] \models \phi$

11/22

Equality

Equality is a **special** predicate

$t_1 = t_2$ is true under a given interpretation
if and only if
 t_1 and t_2 refer to the same constant

That is,

$$\mathcal{I}, \nu \models t_1 = t_2 \iff \nu(t_1) = \nu(t_2)$$

12/22

Examples

- Let the set of **constants** be $\{\text{John, Mary, Jane, Scooby}\} \cup \mathbb{N}$
- Consider the **predicates** $\text{Person}(\cdot, \cdot)$ and $\text{Happy}(\cdot)$
- Take the **semantic function** $\mathcal{I}$ such that

$$\text{Person}^{\mathcal{I}} = \{(\text{John}, 24), (\text{Jane}, 20), (\text{Mary}, 26)\}$$

$$\text{Happy}^{\mathcal{I}} = \{\text{Scooby, Jane, Mary}\}$$

Is there an **assignment** ν such that $(\mathcal{I}, \nu)$ **satisfies**

- $\text{Happy}(x) \wedge \neg \exists y \text{ Person}(x, y)$?
- $\exists x, y \text{ Person}(x, z) \wedge \text{Person}(y, z)$?
- $\exists x, y \text{ Person}(x, z) \wedge \text{Person}(y, z) \wedge \neg(x = y)$?
- $\forall x \text{ Happy}(x) \rightarrow \exists y \text{ Person}(x, y)$?

13/22

Satisfiability and validity

An interpretation $(\mathcal{I}, \nu)$ is a **model** of ϕ if $\mathcal{I}, \nu \models \phi$

A formula is

satisfiable if it has a model

unsatisfiable if it has no models

falsifiable if there is some interpretation that is not a model

valid (i.e., a **tautology**) if every interpretation is a model

14/22

Equivalence

Two formulas are **logically equivalent** (written $\phi \equiv \psi$) if they have the same models

That is, for all interpretations $(\mathcal{I}, \nu)$

$$\mathcal{I}, \nu \models \phi \iff \mathcal{I}, \nu \models \psi$$

Questions:

- Are $P(x)$ and $P(y)$ logically equivalent?
- What about $\forall x P(x)$ and $\forall y P(y)$?

15/22

Universal quantification

Everyone taking IDB is smart:

$$\forall x \left(\text{Takes}(x, \text{idb}) \rightarrow \text{Smart}(x) \right)$$

Typically $\rightarrow$ is the main connective with $\forall$

Common mistake: using $\wedge$ as the main connective with $\forall$:

$$\forall x \left(\text{Takes}(x, \text{idb}) \wedge \text{Smart}(x) \right)$$

means “Everyone takes IDB, and everyone is smart”

16/22

Existential quantification

Someone takes IDB and fails:

$$\exists x \left(\text{Takes}(x, \text{idb}) \wedge \text{Fails}(x, \text{idb}) \right)$$

Typically $\wedge$ is the main connective with $\exists$

Common mistake: using $\rightarrow$ as the main connective with $\exists$:

$$\exists x \left(\text{Takes}(x, \text{idb}) \rightarrow \text{Fails}(x, \text{idb}) \right)$$

is true if there is anyone who does not take IDB

17/22

Properties of quantifiers

- $\forall x \forall y \phi$ is the same as $\forall y \forall x \phi$
- $\exists x \exists y \phi$ is the same as $\exists y \exists x \phi$
- $\exists x \forall y \phi$ is **not the same** as $\forall y \exists x \phi$

Example

$$\exists x \forall y \text{ Loves}(x, y)$$

means “There is somebody who loves everyone in the world”

$$\forall y \exists x \text{ Loves}(x, y)$$

means “Everyone is loved by somebody (not necessarily the same)”

18/22

Quantifier duality

Each can be expressed using the other:

$$\forall x \text{ Likes}(x, \text{cake}) \equiv \neg \exists x \neg \text{ Likes}(x, \text{cake})$$

Everybody likes cakes is the same as saying

There is not anybody who does not like cake

$$\exists x \text{ Likes}(x, \text{broccoli}) \equiv \neg \forall x \neg \text{ Likes}(x, \text{broccoli})$$

Somebody likes broccoli is the same as saying

Not everybody does not like broccoli

19/22

Equivalences (1)

Commutativity

$$\phi \vee \psi \equiv \psi \vee \phi$$

$$\phi \wedge \psi \equiv \psi \wedge \phi$$

Associativity

$$(\phi \vee \psi) \vee \chi \equiv \phi \vee (\psi \vee \chi)$$

$$(\phi \wedge \psi) \wedge \chi \equiv \phi \wedge (\psi \wedge \chi)$$

Distributivity

$$\phi \wedge (\psi \vee \chi) \equiv (\phi \wedge \psi) \vee (\phi \wedge \chi)$$

$$\phi \vee (\psi \wedge \chi) \equiv (\phi \vee \psi) \wedge (\phi \vee \chi)$$

Idempotence

$$\phi \vee \phi \equiv \phi$$

$$\phi \wedge \phi \equiv \phi$$

Absorption

$$\phi \vee (\phi \wedge \psi) \equiv \phi$$

$$\phi \wedge (\phi \vee \psi) \equiv \phi$$

20/22

Equivalences (2)

Double Negation

$$\neg\neg\phi \equiv \phi$$

De Morgan

$$\neg(\phi \vee \psi) \equiv \neg\phi \wedge \neg\psi$$

$$\neg(\phi \wedge \psi) \equiv \neg\phi \vee \neg\psi$$

Implication

$$\phi \rightarrow \psi \equiv \neg\phi \vee \psi$$

21/22

Equivalences (3)

$$(\forall x \phi) \wedge \psi \equiv \forall x (\phi \wedge \psi) \quad \text{if } x \text{ is not free in } \psi$$

$$(\forall x \phi) \vee \psi \equiv \forall x (\phi \vee \psi) \quad \text{if } x \text{ is not free in } \psi$$

$$(\exists x \phi) \wedge \psi \equiv \exists x (\phi \wedge \psi) \quad \text{if } x \text{ is not free in } \psi$$

$$(\exists x \phi) \vee \psi \equiv \exists x (\phi \vee \psi) \quad \text{if } x \text{ is not free in } \psi$$

$$(\forall x \phi) \wedge (\forall x \psi) \equiv \forall x (\phi \wedge \psi)$$

$$(\exists x \phi) \vee (\exists x \psi) \equiv \exists x (\phi \vee \psi)$$

$$\neg \forall x \phi \equiv \exists x \neg \phi$$

$$\neg \exists x \phi \equiv \forall x \neg \phi$$

Introduction to Databases

(INFR10080)

(Relational Calculus)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

First-order logic

term $t := x$ (variable)
 $| c$ (constant)

formula $\varphi := P(t_1, \dots, t_n)$
 $| t_1 \text{ op } t_2$ with $\text{op} \in \{=, \neq, >, <, \geq, \leq\}$
 $| \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \neg \varphi \mid \varphi_1 \rightarrow \varphi_2$
 $| \exists x \varphi \mid \forall x \varphi$ if $x \in \text{free}(\varphi)$

$\text{free}(\varphi)$ = variables that are not in the scope of any quantifier

2/12

Notation

We write $\exists x_1 \exists x_2 \dots \exists x_n \varphi$ as $\exists x_1, \dots, x_n \varphi$
and $\forall x_1 \forall x_2 \dots \forall x_n \varphi$ as $\forall x_1, \dots, x_n \varphi$

We assume **quantifiers bind till the end** of the formula:

Example

$\exists x R(x) \wedge S(x)$ stands for $\exists x (R(x) \wedge S(x))$
not for $(\exists x R(x)) \wedge S(x)$

3/12

Relational calculus

A **relational calculus query** is an expression of the form $\{\bar{x} \mid \varphi\}$ where the set of variables in $\bar{x}$ is **free**(φ)

Examples

- $Q = \{x, y \mid \exists z R(x, z) \wedge S(z, y)\}$
- $Q = \{y, x \mid \exists z R(x, z) \wedge S(z, y)\}$
- $Q = \{x, x \mid \forall y R(x, y)\}$

Queries without free variables are called **Boolean queries**

Examples

- $Q = \{() \mid \forall x R(x, x)\}$
- $Q = \{() \mid \forall x \exists y R(x, y)\}$

4/12

Data model

Relations (tables) are **sets** of **tuples** of the same length

Schema

- Set of **relation names**
- **Arity** (i.e., number of columns) of each relation name
Note that columns are ordered but have no names

Instance

- Each relation name (from the schema) of arity k is associated with a k -ary relation
(i.e., a set of tuples that are all of length k)

5/12

Examples

Customer : ID, Name, Age

Account : Number, Branch, CustID

Q₁: Name of customers younger than 33 or older than 50

$$\{ y \mid \exists x, z \text{ Customer}(x, y, z) \wedge (z < 33 \vee z > 50) \}$$

Q₂: Name and age of customers having an account in London

$$\{ y, z \mid \exists x \text{ Customer}(x, y, z) \wedge \exists w \text{ Account}(w, \text{'London'}, x) \}$$

*Q₃: ID of customers who have an account in **every** branch*

$$\{ x \mid \exists y, z \text{ Customer}(x, y, z) \wedge (\forall u, w, v \text{ Account}(u, w, v) \rightarrow \exists u' \text{ Account}(u', w, x)) \}$$

6/12

Answers to queries

A **database instance** can be seen as a **semantic function** in FOL
(mapping relation names to relations of appropriate arity)

Recall An **assignment** ν maps variables to constants
(and each constant to itself)

Notation

For $\bar{x} = (x_1, x_2, \dots, x_n)$, $\nu(\bar{x}) = (\nu(x_1), \nu(x_2), \dots, \nu(x_n))$

The **answer** to a query $Q = \{ \bar{x} \mid \varphi \}$ on a database D is

$$Q(D) = \{ \nu(\bar{x}) \mid \nu \text{ is an assignment for which } D, \nu \models \varphi \}$$

The answer to a **Boolean query** is either $\{ () \}$ (**true**) or $\emptyset$ (**false**)

7/12

Safety

A query is **safe** if it gives a **finite answer** on **all** databases and this answer does **not depend on the underlying set of constants**

Examples of unsafe queries:

- $\{x \mid \neg R(x)\}$
- $\{x, y \mid R(x) \vee R(y)\}$
- $\{x, y \mid x = y\}$

Question: Are Boolean queries safe?

Bad news

Whether a relational calculus query is safe is **undecidable**

8/12

Active domain

$\mathbf{adom}(R) = \{ \text{all constants occurring in } R \}$

Example

$$\mathbf{adom} \left(\begin{array}{c|cc} R & A & B \\ \hline & a_1 & b_1 \\ & a_1 & b_2 \end{array} \right) = \{a_1, b_1, b_2\}$$

The active domain of a **database** D is

$$\mathbf{adom}(D) = \bigcup_{R \in D} \mathbf{adom}(R)$$

9/12

Active domain semantics

Evaluate queries within $\mathbf{adom}(D) \Rightarrow$ **safe relational calculus**

$$Q(D) = \{ \nu(\bar{x}) \mid \nu \text{ is an assignment of constants in } \mathbf{adom}(D) \\ \text{for which } D, \nu \models \varphi \}$$

Interpretation of formulas under (D, ν)

$$\begin{array}{ll} D, \nu \models P(t_1, \dots, t_n) & \iff (\nu(t_1), \dots, \nu(t_n)) \in P^D \\ D, \nu \models \neg \varphi & \iff D, \nu \not\models \varphi \\ D, \nu \models \varphi \wedge \psi & \iff D, \nu \models \varphi \text{ and } D, \nu \models \psi \\ D, \nu \models \varphi \vee \psi & \iff D, \nu \models \varphi \text{ or } D, \nu \models \psi \\ D, \nu \models \varphi \rightarrow \psi & \iff \text{if } D, \nu \models \varphi \text{ then } D, \nu \models \psi \\ D, \nu \models \forall x \varphi & \iff \text{for every } c \in \mathbf{adom}(D): D, \nu[x/c] \models \varphi \\ D, \nu \models \exists x \varphi & \iff \text{there is } c \in \mathbf{adom}(D) \text{ s.t. } D, \nu[x/c] \models \varphi \end{array}$$

10/12

Evaluation of quantifiers

$$D, \nu \models \exists x \varphi \iff D, \nu \models \bigvee_{a \in \mathbf{adom}(D)} \varphi[x/a]$$

$$D, \nu \models \forall x \varphi \iff D, \nu \models \bigwedge_{a \in \mathbf{adom}(D)} \varphi[x/a]$$

where $\varphi[x/a]$ denotes the formula obtained from φ
by replacing all **free** occurrences of x with a

11/12

Evaluation of quantifiers: Examples

Assume $\mathbf{adom}(D) = \{1, 2, 3\}$

$$D, \nu \models \exists x R(x, y) \wedge S(x)$$

$$\iff$$

$$D, \nu \models (R(1, y) \wedge S(1)) \vee (R(2, y) \wedge S(2)) \vee (R(3, y) \wedge S(3))$$

$$D, \nu \models \forall x S(x) \rightarrow R(x, y)$$

$$\iff$$

$$D, \nu \models (S(1) \rightarrow R(1, y)) \wedge (S(2) \rightarrow R(2, y)) \wedge (S(3) \rightarrow R(3, y))$$

Introduction to Databases

(INFR10080)

(Equivalence of RA and RC)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Algebra $\equiv$ Safe calculus

Fundamental theorem of database theory:

Relational algebra and Safe relational calculus **equally expressive**

- For every query in safe relational calculus there exists an equivalent query in relational algebra
- For every query in relational algebra there exists an equivalent query in safe relational calculus

We need a **correspondence** between column **names** and **positions**

When we say that a **base relation** R is over attributes $A, B, C, \dots$ this means the 1st column is A , the 2nd is B , the 3rd is C , etc.

2/17

From algebra to calculus

Translate each RA expression E into a FOL formula φ

Assumptions (without loss of generality)

- Each renaming operation renames **only one** attribute

Environment η

Injective map from attributes to variables

Unless stated otherwise, for an attribute A we assume $\eta(A) = x_A$

But in general the chosen **variable names can be arbitrary**

3/17

From algebra to calculus

Base relation

R over $A_1, \dots, A_n$ is translated to $R(\eta(A_1), \dots, \eta(A_n))$

Example

If R is a base relation over A, B

$$\eta = \{ A \mapsto x_A, B \mapsto x_B, \dots \}$$

then R is translated to $R(x_A, x_B)$

4/17

From algebra to calculus

Renaming $\rho_{\text{old} \rightarrow \text{new}}(E)$

1. Translate E to φ
2. If there is no mapping for **new** in η , add $\{\text{new} \mapsto x_{\text{new}}\}$
3. Replace every occurrence of $\eta(\text{new})$ in φ with a **fresh** variable
4. Replace every (free) occurrence of $\eta(\text{old})$ in φ by $\eta(\text{new})$

Example

If R is a base relation over A, B

then $\rho_{A \rightarrow B}(\rho_{B \rightarrow C}(R))$ is translated to $R(x_B, x_C)$

5/17

From algebra to calculus

Projection

$\pi_\alpha(E)$ is translated to $\exists X \varphi$

where

- φ is the translation of E
- $X = \mathbf{free}(\varphi) - \eta(\alpha)$
(attributes that are **not** projected become quantified)

Example

If R is a base relation over A, B

then $\pi_A(R)$ is translated into $\exists x_B R(x_A, x_B)$

6/17

From algebra to calculus

Selection

$\sigma_\theta(E)$ is translated to $\varphi \wedge \eta(\theta)$

where

- φ is the translation of E
- $\eta(\theta)$ is obtained from θ by replacing each attribute A by $\eta(A)$

Example

If R is a base relation over A, B

then $\sigma_{A=B}(R)$ is translated into $R(x_A, x_B) \wedge x_A = x_B$

7/17

From algebra to calculus

Product	$E_1 \times E_2$	is translated to	$\varphi_1 \wedge \varphi_2$
Union	$E_1 \cup E_2$	is translated to	$\varphi_1 \vee \varphi_2$
Difference	$E_1 - E_2$	is translated to	$\varphi_1 \wedge \neg \varphi_2$

where

- φ_1 is the translation of E_1
- φ_2 is the translation of E_2

8/17

Example

Customer : CustID, Name

Account : Number, CustID

Environment $\eta = \{ \text{CustID} \mapsto x_1, \text{Name} \mapsto x_2, \text{Number} \mapsto x_3 \}$

How do we translate **Customer** $\bowtie$ **Account** ? Blackboard time!

$$\exists x_4 \text{ Customer}(x_1, x_2) \wedge \text{Account}(x_3, x_4) \wedge x_1 = x_4$$

9/17

Active domain in relational algebra

For R over attributes $A_1, \dots, A_n$

$\mathbf{adom}(R)$ is given by $\rho_{A_1 \rightarrow A}(\pi_{A_1}(R)) \cup \dots \cup \rho_{A_n \rightarrow A}(\pi_{A_n}(R))$

$$\mathbf{adom}(D) = \bigcup_{R \in D} \mathbf{adom}(R)$$

We denote by $\mathbf{Adom}_N$ the RA expression that, on a database D , returns a table

- with a single column, named N
- consisting of all elements of $\mathbf{adom}(D)$

10/17

From calculus to algebra

Translate each FOL formula φ into an RA expression E

Assumptions (without loss of generality)

- No universal quantifiers, implications, double negations
- No distinct pair of quantifiers binds the same variable name
- No variable name occurs both free and bound
- No variable name is repeated within a predicate
- No constants in predicates
- No atoms of the form $x \mathbf{op} x$ or $c_1 \mathbf{op} c_2$

Environment η

Injective map from variables to attributes

Unless stated otherwise, for a variable x we assume $\eta(x) = A_x$

But in general the chosen attribute names can be arbitrary

11/17

From calculus to algebra

Let R be over attributes $A_1, \dots, A_n$

Predicate

$R(x_1, \dots, x_n)$ is translated to $\rho_{A_1 \rightarrow \eta(x_1), \dots, A_n \rightarrow \eta(x_n)}(R)$

Example

For R over attributes A, B, C ,

$R(x, y, z)$ is translated into $\rho_{A \rightarrow A_x, B \rightarrow A_y, C \rightarrow A_z}(R)$

12/17

From calculus to algebra

Existential quantification

$\exists x \varphi$ is translated to $\pi_{\eta(X - \{x\})}(E)$

where

- E is the translation of φ
- $X = \text{free}(\varphi)$

Example

For φ with free variables x, y, z and translation E ,

$\exists y \varphi$ is translated to $\pi_{A_x, A_z}(E)$

13/17

From calculus to algebra

Comparisons

$x \text{ op } y$ is translated to $\sigma_{\eta(x) \text{ op } \eta(y)}(\mathbf{Adom}_{\eta(x)} \times \mathbf{Adom}_{\eta(y)})$

$x \text{ op } c$ is translated to $\sigma_{\eta(x) \text{ op } c}(\mathbf{Adom}_{\eta(x)})$

Example

$x = y$ is translated to $\sigma_{A_x = A_y}(\mathbf{Adom}_{A_x} \times \mathbf{Adom}_{A_y})$

$x > 1$ is translated to $\sigma_{A_x > 1}(\mathbf{Adom}_{A_x})$

14/17

From calculus to algebra

Negation

$\neg\varphi$ is translated into $\left(\bigtimes_{x \in \text{free}(\varphi)} \mathbf{Adom}_{\eta(x)} \right) - E$

where E is the translation of φ

Example

For φ with free variables x, y and translation E

$\neg\varphi$ is translated to $\mathbf{Adom}_{A_x} \times \mathbf{Adom}_{A_y} - E$

15/17

From calculus to algebra

Disjunction: $\varphi_1 \vee \varphi_2$ is translated to

$$E_1 \times \left(\bigtimes_{x \in X_2 - X_1} \mathbf{Adom}_{\eta(x)} \right) \cup E_2 \times \left(\bigtimes_{x \in X_1 - X_2} \mathbf{Adom}_{\eta(x)} \right)$$

where, for $i \in \{1, 2\}$,

- E_i is the translation of φ_i
- $X_i = \mathbf{free}(\varphi_i)$

Conjunction: same as disjunction, but use $\cap$ instead of $\cup$

16/17

Example

Customer : CustID, Name

Account : Number, CustID

Translate $\exists x_4 \text{ Customer}(x_1, x_2) \wedge \text{Account}(x_3, x_4) \wedge x_1 = x_4$

Environment $\eta = \{ x_1 \mapsto A, x_2 \mapsto B, x_3 \mapsto C, x_4 \mapsto D \}$

$$\pi_{A,B,C} \left(\left(E_1 \times \mathbf{Adom}_C \times \mathbf{Adom}_D \right) \cap \left(\mathbf{Adom}_A \times \mathbf{Adom}_B \times E_2 \right) \cap \left(\sigma_{A=D}(\mathbf{Adom}_A \times \mathbf{Adom}_D) \times \mathbf{Adom}_B \times \mathbf{Adom}_C \right) \right)$$

where

- $E_1 = \rho_{\text{CustID} \rightarrow A, \text{Name} \rightarrow B}(\text{Customer})$
- $E_2 = \rho_{\text{Number} \rightarrow C, \text{CustID} \rightarrow D}(\text{Account})$

17/17

Introduction to Databases

(INFR10080)

(Multisets)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Duplicates

R	$\pi_A(R)$	$\text{SELECT } A \text{ FROM } R$
A B	A	A
a1 b1	a1	a1
a2 b2	a2	a2
a1 b2		a1

- We considered relational algebra on **sets**
- SQL uses **bags**: sets with duplicates

2/13

Multisets (a.k.a. bags)

Sets where the **same element** can occur **multiple times**

The **number of occurrences** of an element is called its **multiplicity**

Notation

$a \in_k B$: a occurs k times in bag B

$a \in B$: a occurs in B with multiplicity ≥ 1

$a \notin B$: a does not occur in B (that is, $a \in_0 B$)

3/13

Relational algebra on bags

Relations are **bags of tuples**

Projection

Keeps duplicates

$$\pi_A \left(\begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 2 & 3 \\ 1 & 1 \\ 2 & 2 \end{array} \right) = \begin{array}{c} \mathbf{A} \\ \hline 2 \\ 1 \\ 2 \end{array}$$

4/13

Relational algebra on bags

Cartesian product

Concatenates tuples as many times as they occur

$$\begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 1 & 1 \end{array} \times \begin{array}{c} \mathbf{C} \\ \hline 2 \\ 2 \end{array} = \begin{array}{ccc} \mathbf{A} & \mathbf{B} & \mathbf{C} \\ \hline 1 & 1 & 2 \\ 1 & 1 & 2 \end{array}$$

5/13

Relational algebra on bags

Selection

Takes all occurrences of tuples satisfying the condition:

$$\text{If } \bar{a} \in_k R, \quad \text{then } \begin{cases} \bar{a} \in_k \sigma_\theta(R) & \text{if } \bar{a} \text{ satisfies } \theta \\ \bar{a} \notin \sigma_\theta(R) & \text{otherwise} \end{cases}$$

Example

$$\sigma_{A>1} \left(\begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 2 & 3 \\ 1 & 2 \\ 2 & 3 \end{array} \right) = \begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 2 & 3 \\ 2 & 3 \end{array}$$

6/13

Relational algebra on bags

Duplicate elimination ε

New operation that removes duplicates:

$$\text{If } \bar{a} \in R, \quad \text{then } \bar{a} \in_1 \varepsilon(R)$$

Example

$$\varepsilon \left(\begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 2 & 3 \\ 1 & 2 \\ 2 & 3 \end{array} \right) = \begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \hline 2 & 3 \\ 1 & 2 \end{array}$$

7/13

Relational algebra on bags

Union

Adds multiplicities:

$$\text{If } \bar{a} \in_k R \text{ and } \bar{a} \in_n S, \quad \text{then } \bar{a} \in_{k+n} R \cup S$$

Example

A	B		A	B		A	B
1	2		1	2		1	2
1	2	$\cup$	1	3	$=$	1	2
1	3		1	4		1	3
						1	2
						1	3
						1	4

8/13

Relational algebra on bags

Intersection

Takes the **minimum** multiplicity:

$$\text{If } \bar{a} \in_k R \text{ and } \bar{a} \in_n S, \quad \text{then } \bar{a} \in_{\min\{k,n\}} R \cap S$$

Example

A	B		A	B		A	B
1	2		1	2		1	2
1	2	$\cap$	1	3	$=$	1	3
1	3		1	4			

9/13

Relational algebra on bags

Difference

Subtracts multiplicities up to zero:

$$\text{If } \bar{a} \in_k R \text{ and } \bar{a} \in_n S, \text{ then } \begin{cases} \bar{a} \in_{k-n} R - S & \text{if } k > n \\ \bar{a} \notin R - S & \text{otherwise} \end{cases}$$

Example

A	B		A	B		A	B
1	2	−	1	2	=	1	2
1	2		1	3			
1	3		1	3			
			1	4			

10/13

RA on sets vs. RA on bags

Equivalences of RA on sets **do not** necessarily hold on **bags**

Example

On bags $\sigma_{\theta_1 \vee \theta_2}(R) \not\equiv \sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R)$

R	A	$\sigma_{A>1 \vee A<3}(R)$	A	$\sigma_{A>1}(R) \cup \sigma_{A<3}(R)$	A
	2		2		2
					2

$\varepsilon(\sigma_{\theta_1 \vee \theta_2}(R)) \equiv \varepsilon(\sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R))$ holds

11/13

Basic SQL queries revisited

$Q :=$ **SELECT** [**DISTINCT**] α **FROM** τ **WHERE** θ
| Q_1 **UNION** [**ALL**] Q_2
| Q_1 **INTERSECT** [**ALL**] Q_2
| Q_1 **EXCEPT** [**ALL**] Q_2

12/13

SQL and RA on bags

SQL

SELECT $\alpha \dots$

SELECT DISTINCT $\alpha \dots$

Q_1 **UNION ALL** Q_2

Q_1 **INTERSECT ALL** Q_2

Q_1 **EXCEPT ALL** Q_2

Q_1 **UNION** Q_2

Q_1 **INTERSECT** Q_2

Q_1 **EXCEPT** Q_2

RA on bags

$\pi_\alpha(\cdot)$

$\varepsilon(\pi_\alpha(\cdot))$

$Q_1 \cup Q_2$

$Q_1 \cap Q_2$

$Q_1 - Q_2$

$\varepsilon(Q_1 \cup Q_2)$

$\varepsilon(Q_1 \cap Q_2)$

$\varepsilon(Q_1) - Q_2$

13/13

Introduction to Databases

(INFR10080)

(Aggregation)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Duplicates and aggregation (1)

Customer			
ID	Name	City	Age
1	John	Edinburgh	31
2	Mary	London	37
3	Jane	London	22
4	Jeff	Cardiff	22

Average age of customers: $\text{avg}(\pi_{\text{Age}}(\text{Customer}))$

- If we remove duplicates we get $\frac{31+37+22}{3} = 30$ (**wrong**)

SQL keeps duplicates by default:

```
SELECT AVG(age)
FROM Customer ;
```

2/20

Duplicates and aggregation (2)

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

Number of branches: $|\varepsilon(\pi_{\text{Branch}}(\text{Account}))|$

- If we keep duplicates we get 3 (**wrong**)

In SQL:

```
SELECT COUNT(DISTINCT branch)
FROM Account ;
```

3/20

Aggregate functions in SQL

COUNT number of elements in a column

AVG average value of elements in a column

SUM adds up all elements in a column

MIN minimum value of elements in a column

MAX maximum value of elements in a column

- Using **DISTINCT** with **MIN** and **MAX** makes no difference
- **COUNT**(*) counts all rows in a table
- **COUNT**(**DISTINCT** *) is **illegal**

To count all **distinct** rows of a table *T* use

```
SELECT COUNT (DISTINCT T.*)  
FROM    T ;
```

4/20

Aggregation and empty tables

Suppose table *T* has a column (of numbers) called *A*

```
SELECT MIN (A) , MAX (A) , AVG (A) , SUM (A) , COUNT (A) , COUNT (*)  
FROM    T  
WHERE    1=2 ;
```

min	max	sum	avg	count	count
				0	0

5/20

Aggregation with grouping

Account			
Num	Branch	CID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

How much money does **each** customer have in total **across all of his accounts**?

Idea

1. Partition **Account** into **groups** (one per customer) of rows
2. Sum balances in each group separately
3. Take the union of the results for each group

6/20

Account			
Num	Branch	CID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

Num	Branch	CID	Balance
111	London	1	1330.00
333	Edinburgh	1	450.00

		CID	SUM
		1	1780.00

Num	Branch	CID	Balance
222	London	2	1756.00

		CID	SUM
		2	1756.00

		CID	SUM
		1	1780.00
		2	1756.00

7/20

Grouping in SQL

Account

Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

How much money does **each** customer have in total **across all of his accounts**?

```
SELECT    A.custid, SUM(A.balance)
FROM      Account A
GROUP BY  A.custid ;
```

Answer:

CustID	SUM
1	1780.00
2	1756.00

8/20

Account

Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

CustID	Balance
1	1330.00
1	450.00

CustID	Balance
2	1756.00

CustID	SUM
1	1780.00

CustID	SUM
2	1756.00

CustID	SUM
1	1780.00
2	1756.00

9/20

Grouping in SQL: Another example

Account

Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

How much money is there in **total in each branch**?

```
SELECT  A.branch, SUM(A.balance)
FROM    Account A
GROUP BY A.branch ;
```

Answer:

Branch	SUM
London	3086.00
Edinburgh	450.00

10/20

Account

Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

Branch	Balance
London	1330.00
London	1756.00

Branch	Balance
Edinburgh	450.00

Branch	SUM
London	3086.00

Branch	SUM
Edinburgh	450.00

Branch	SUM
London	3086.00
Edinburgh	450.00

11/20

YAGE: Yet Another Grouping Example

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00
444	Edinburgh	2	613.00
555	London	1	300.00
666	Edinburgh	2	1217.00

How much money does **each customer** have in **each branch**?

```
SELECT  A.custid, A.branch, SUM(A.balance)
FROM    Account A
GROUP BY A.custid, A.branch ;
```

Answer:

CustID	Branch	SUM
1	London	1630.00
1	Edinburgh	450.00
2	London	1756.00
2	Edinburgh	1830.00

12/20

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00
444	Edinburgh	2	613.00
555	London	1	300.00
666	Edinburgh	2	1217.00

Branch	CustID	Balance
London	1	1330.00
London	1	300.00

Branch	CustID	Balance
London	2	1756.00

Branch	CustID	Balance
Edinburgh	1	450.00

Branch	CustID	Balance
Edinburgh	2	613.00
Edinburgh	2	1217.00

Branch	CustID	SUM
London	1	1630.00

Branch	CustID	SUM
London	2	1756.00

Branch	CustID	SUM
Edinburgh	1	450.00

Branch	CustID	SUM
Edinburgh	2	1830.00

CustID	Branch	SUM
1	London	1630.00
1	Edinburgh	450.00
2	London	1756.00
2	Edinburgh	1830.00

13/20

Beware

Without constraints, in queries with **GROUP BY**, attributes in **SELECT** **must**

- appear in the **GROUP BY**
- or**
- be used in an aggregate function

The following query gives an error:

```
SELECT    A.custid, A.branch, SUM(A.balance)
FROM      Account A
GROUP BY  A.branch ;
```

14/20

Filtering based on aggregation

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

Branches with a total balance (across accounts) of at least 500?

```
SELECT    A.branch, SUM(A.balance)
FROM      Account A
GROUP BY  A.branch
HAVING    SUM(A.balance) >= 500 ;
```

Answer:	Branch	SUM
	London	3086.00

15/20

Order of evaluation

1. Take rows from the (joined) tables listed in **FROM**
2. Discard rows not satisfying the **WHERE** condition
3. Partition rows according to attributes in **GROUP BY**
4. **Compute aggregates**
5. Discard rows not satisfying the **HAVING** condition
6. Output the values of expressions listed in **SELECT**

16/20

This frame intentionally left blank.

Aggregation and arithmetic (1)

Account

Number	Branch	CustID	Balance	Spend
111	London	1	1330.00	250.00
222	London	2	1756.00	356.00
333	Edinburgh	1	450.00	0.00

Money available in total to each customer across his accounts

```
SELECT  A.custid, SUM(A.balance - A.spend)
FROM    Account A
GROUP BY A.custid ;
```

Answer:

CustID	SUM
1	1530.00
2	1400.00

17/20

Account

Number	Branch	CustID	Balance	Spend
111	London	1	1330.00	250.00
222	London	2	1756.00	356.00
333	Edinburgh	1	450.00	0.00

CustID	Balance	Spend	CustID	Balance	Spend
1	1330.00	250.00	2	1756.00	356.00
1	450.00	0.00			
CustID	Balance – Spend		CustID	Balance – Spend	
1	1080.00		2	1400.00	
1	450.00				
CustID	SUM		CustID	SUM	
1	1530.00		2	1400.00	
CustID	SUM				
1	1530.00		2	1400.00	

18/20

Aggregation and arithmetic (2)

Account

Number	Branch	CustID	Balance	Spend
111	London	1	1330.00	250.00
222	London	2	1756.00	356.00
333	Edinburgh	1	450.00	0.00

Money available in total to each customer across his accounts

```
SELECT  A.custid, SUM(A.balance) - SUM(A.spend)
FROM    Account A
GROUP BY A.custid ;
```

Answer:

CustID	?column?
1	1530.00
2	1400.00

19/20

Account

Number	Branch	CustID	Balance	Spend
111	London	1	1330.00	250.00
222	London	2	1756.00	356.00
333	Edinburgh	1	450.00	0.00

CustID	Balance	Spend	CustID	Balance	Spend
1	1330.00	250.00	2	1756.00	356.00
1	450.00	0.00			
CustID	SUM	SUM	CustID	SUM	SUM
1	1780.00	250.00	2	1756.00	356.00
CustID	?column?		CustID	?column?	
1	1530.00		2	1400.00	

CustID	?column?
1	1530.00
2	1400.00

20/20

Introduction to Databases

(INFR10080)

(Nested Queries)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Aggregate results in WHERE

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

Accounts with a **higher balance** than the **average of all accounts**

```
SELECT A.number
FROM Account A
WHERE A.balance > ( SELECT AVG (A1.balance)
                   FROM Account A1 );
```

Answer:

Number
111
222

2/20

Aggregate results in WHERE

Accounts with a higher balance than the average of all accounts

```
SELECT A.number
FROM Account A
WHERE A.balance > AVG ( SELECT A1.balance
                       FROM Account A1 );
```

ERROR

Aggregate functions can only be used in **SELECT** and **HAVING**

3/20

Comparisons with subquery results

```
SELECT ...  
FROM ...  
WHERE term op ( subquery ) ;
```

Allowed as long as **subquery** returns a **single value**

```
SELECT ...  
FROM ...  
WHERE (term1, ..., termn) op ( subquery ) ;
```

Allowed as long as **subquery** returns a **single row** with n columns

4/20

The WHERE clause revisited

term := attribute | value

comparison :=

- (term, ..., term) **op** (term, ..., term)
with **op** ∈ {=, <>, <, >, <=, >=}
- term **IS** [**NOT**] **NULL**
- (term, ..., term) **op** **ANY** (query)
- (term, ..., term) **op** **ALL** (query)
- (term, ..., term) [**NOT**] **IN** (query)
- **EXISTS** (query)

condition :=

- comparison
- condition **AND** condition
- condition **OR** condition
- **NOT** condition

5/20

Comparisons between tuples

Equality

$$(t_1, \dots, t_n) = (t'_1, \dots, t'_n) \iff t_1 = t'_1 \text{ AND } \dots \text{ AND } t_n = t'_n$$

Less-than

$$(t_1, t_2, \dots, t_n) < (t'_1, t'_2, \dots, t'_n) \iff t_1 < t'_1 \text{ OR } (t_1 = t'_1 \text{ AND } (t_2, \dots, t_n) < (t'_2, \dots, t'_n))$$

Others (derived)

For $\bar{t} = (t_1, \dots, t_n)$ and $\bar{t}' = (t'_1, \dots, t'_n)$:

$\bar{t} < > \bar{t}'$	$\iff$	$\text{NOT } (\bar{t} = \bar{t}')$
$\bar{t} < = \bar{t}'$	$\iff$	$\bar{t} < \bar{t}' \text{ OR } \bar{t} = \bar{t}'$
$\bar{t} > \bar{t}'$	$\iff$	$\text{NOT } (\bar{t} < = \bar{t}')$
$\bar{t} > = \bar{t}'$	$\iff$	$\text{NOT } (\bar{t} < \bar{t}')$

6/20

ANY

$(\text{term}, \dots, \text{term}) \text{ op ANY (query)}$

True if **there exists** a row $\bar{r}$ in the results of **query** such that $(\text{term}, \dots, \text{term}) \text{ op } \bar{r}$ is true

Examples:

Consider the table T =

A
1
2
3

- $3 < \text{ANY}(\text{SELECT } A \text{ FROM } T)$ is false
- $3 < \text{ANY}(\text{SELECT } A+1 \text{ FROM } T)$ is true
- What about $3 < \text{ANY}(\text{SELECT } A \text{ FROM } T \text{ WHERE } A = 0)$?

7/20

ALL

(term, ..., term) **op** ALL (query)

True if **for all** rows $\bar{r}$ in the results of query

(term, ..., term) **op** $\bar{r}$ is true

Examples:

Consider the table T =

A
3
4
5
6

- $3 < \text{ALL}(\text{SELECT } A \text{ FROM } T \text{ WHERE } A \neq 3)$ is true
- $3 < \text{ALL}(\text{SELECT } A \text{ FROM } T \text{ WHERE } A \neq 6)$ is false
- What about $3 < \text{ALL}(\text{SELECT } A \text{ FROM } T \text{ WHERE } A = 0)$?

8/20

Examples with ANY / ALL

Customer: *ID, Name, City*

Account: *Number, Branch, CustID, Balance*

ID of customers from London who own an account

```
SELECT C.id
FROM Customer C
WHERE C.city = 'London'
AND C.id = ANY ( SELECT A.custid
                  FROM Account A );
```

Customers living in cities without a branch

```
SELECT *
FROM Customer C
WHERE C.city <> ALL ( SELECT A.branch
                     FROM Account A );
```

9/20

IN / NOT IN

(term, ..., term) **IN** (query)

same as

(term, ..., term) = **ANY** (query)

(term, ..., term) **NOT IN** (query)

same as

(term, ..., term) <> **ALL** (query)

10/20

Examples with IN / NOT IN

ID of customers from London who own an account

```
SELECT C.id
FROM   Customer C
WHERE  C.city = 'London'
      AND C.id IN ( SELECT A.custid
                     FROM   Account A );
```

Customers living in cities without a branch

```
SELECT *
FROM   Customer C
WHERE  C.city NOT IN ( SELECT A.branch
                       FROM   Account A );
```

11/20

EXISTS

EXISTS (query) is true if the result of query is **non-empty**

(Stupid) Example

Return all the customers if there are some accounts in London

```
SELECT *  
FROM Customer  
WHERE EXISTS ( SELECT 1  
                  FROM Account  
                  WHERE branch = 'London' );
```

12/20

Correlated subqueries

All nested queries can refer to attributes in the **parent queries**

(Smarter) Example

Return customers who have an account in London

```
SELECT *  
FROM Customer C  
WHERE EXISTS ( SELECT 1  
                  FROM Account A  
                  WHERE A.branch = 'London'  
                  AND A.custid = C.id );
```

parameters = attributes of a subquery that refer to outer queries

13/20

Examples with EXISTS / NOT EXISTS

ID of customers from London who own an account

```
SELECT C.id
FROM Customer C
WHERE C.city = 'London'
AND EXISTS ( SELECT *
              FROM Account A
              WHERE A.custid = C.id);
```

Customers living in cities without a branch

```
SELECT *
FROM Customer C
WHERE NOT EXISTS ( SELECT *
                   FROM Account A
                   WHERE A.branch = C.city );
```

14/20

Scoping

A subquery has

- a **local scope** (its **FROM** clause)
- ***n* outer scopes** (where *n* is the **level of nesting**)
(these are the **FROM** clauses of the parent queries)

For each **reference** to an attribute

1. Look for a binding in the local scope
2. If no binding is found, look in the **closest** outer scope
3. If no binding is found, look in the next closest outer scope
4. ...
5. If no binding is found, give error

15/20

Attribute bindings

```
SELECT *  
FROM table1  
WHERE EXISTS ( SELECT 1  
                FROM table2  
                WHERE A = B );
```

What A , B refer to depends on the attributes in `table1` and `table2`

- Always give aliases to tables
- Always prefix the attributes with the tables they refer to

```
SELECT *  
FROM table1 T1  
WHERE EXISTS ( SELECT 1  
                FROM table2 T2  
                WHERE T2.A = T1.B );
```

16/20

The FROM clause revisited

`FROM table1 [[AS] T1], ..., tablen [[AS] Tn]`

`table :=`

- base-table
- join-table
- (query)

`join-table :=`

- table JOIN table ON condition
- table NATURAL JOIN table
- table CROSS JOIN table

17/20

Subqueries in FROM

Must always be given a name

```
SELECT * FROM ( SELECT * FROM R );
```

ERROR: subquery in FROM must have an alias

Cannot refer to attributes of other tables in the same FROM clause

```
SELECT *  
FROM R, ( SELECT * FROM S WHERE S.a=R.a ) S1 ;
```

ERROR: invalid reference to FROM-clause entry for table "r"

18/20

Example: Avoiding HAVING

Branches with a total balance (across accounts) of at least 500

```
SELECT    A.branch  
FROM      Account A  
GROUP BY A.branch  
HAVING    SUM(A.balance) >= 500 ;
```

Same query without **HAVING**:

```
SELECT subquery.branch  
FROM   ( SELECT    A.branch, SUM(A.balance) AS total  
          FROM      Account A  
          GROUP BY A.branch ) AS subquery  
WHERE  subquery.total >= 500 ;
```

19/20

Example: Aggregation on aggregates

Average of the total balances across each customer's accounts

1. Find the total balance across each customer's accounts
2. Take the average of the totals

```
SELECT      AVG(subquery.tot)
FROM        ( SELECT      A.custid, SUM(A.balance) AS tot
              FROM        Account A
              GROUP BY    A.custid ) AS subquery ;
```

Introduction to Databases

(INFR10080)

(Other Bits of SQL)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Ordering

ORDER BY $\langle \text{column}_1 \rangle$ [**DESC**], ..., $\langle \text{column}_n \rangle$ [**DESC**]

Sorts the output rows according to the values of column_1

If two rows have the same value for column_1 ,
they are sorted by the values of column_2 and so on ...

- Default ordering is **ascending** (can be specified with **ASC**)
- **Descending** ordering is specified by **DESC**

2/8

Ordering example (1)

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

```
SELECT      *  
FROM        Account  
ORDER BY    custid ASC, balance DESC ;
```

Number	Branch	CustID	Balance
111	London	1	1330.00
333	Edinburgh	1	450.00
222	London	2	1756.00

3/8

Ordering example (2)

Account			
Number	Branch	CustID	Balance
111	London	1	1330.00
222	London	2	1756.00
333	Edinburgh	1	450.00

```
SELECT *  
FROM Account  
ORDER BY custid DESC, balance ASC ;
```

Number	Branch	CustID	Balance
222	London	2	1756.00
333	Edinburgh	1	450.00
111	London	1	1330.00

4/8

Casting

CAST(term **AS** $\langle$ type $\rangle$)

Rounding

CAST(102.4675 **AS NUMERIC**(5,2)) gives 102.47

Useful also to produce values in a specific format

Aggregation

AVG(**CAST**(term **AS NUMERIC**(p , s)))

avoids rounding errors in some systems

5/8

Conditional expressions (1)

```
CASE WHEN <bool-expr>  
      THEN <value-expr>  
      . . .  
      WHEN <bool-expr>  
      THEN <value-expr>  
      ELSE <value-expr>  
END
```

- Each **bool-expr** is evaluated in order (from top to bottom)
- When **bool-expr** is true, then the value produced by the corresponding **value-expr** is returned and the **CASE** ends (subsequent **WHEN** are not evaluated)
- If no **WHEN** evaluates to true, the value produced by **value-expr** in **ELSE** is returned
- **ELSE** is optional (if missing, the default value is **NULL**)

6/8

Conditional expressions (2)

Can be used in **SELECT**, **WHERE** and **HAVING**

Return the values of column *A* replacing **NULL** values with 0

```
SELECT CASE WHEN R.A IS NULL THEN 0  
          ELSE R.A END  
FROM R ;
```

Join *R* and *S* on column *A* (don't do this!)

```
SELECT *  
FROM    R, S  
WHERE   CASE WHEN R.A = S.A THEN TRUE  
          ELSE FALSE END ;
```

7/8

Pattern matching

New comparison: term **LIKE** pattern

where **pattern** is a string consisting of

characters (case-sensitive!)

_ (underscore) wildcard matching any one character

% (percent) wildcard matching any substring (including empty)

Example

Customers with a name that **begins with 'K'**

and has **at least 5 characters**

```
SELECT *  
FROM Customer  
WHERE name LIKE 'K____%' ;
```

Introduction to Databases

(INFR10080)

(Database Constraints)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Integrity constraints

Databases often required to satisfy some integrity constraints

Determine what tuples can be stored in the database

Instances that satisfy the constraints are called **legal**

Common constraints: **keys** and **foreign keys**

These are special cases of more general constraints

- **Functional dependencies**
- **Inclusion dependencies**

2/16

Functional dependencies (FDs)

Constraints of the form $X \rightarrow Y$, where X, Y are sets of attributes

Semantics (on sets)

A relation R satisfies $X \rightarrow Y$ if for every two tuples $t_1, t_2 \in R$

$$\pi_X(t_1) = \pi_X(t_2) \implies \pi_Y(t_1) = \pi_Y(t_2)$$

Intuition: The values for the X attributes
determine the values for the Y attributes

Trivial FDs: $X \rightarrow Y$ where $Y \subseteq X$

3/16

Examples of FDs

Employee	Department	Manager
John	Finance	Smith
Mary	HR	Taylor
Susan	HR	Taylor
John	Sales	Smith

Which of the following FDs would the above relation satisfy?

- Department $\rightarrow$ Manager **Yes**
- Employee $\rightarrow$ Department **No**
- Employee, Manager $\rightarrow$ Department **No**
- Manager $\rightarrow$ Department **No**

4/16

Keys

A set of attributes X is a **key** for relation R if for every $t_1, t_2 \in R$

$$\pi_X(t_1) = \pi_X(t_2) \implies t_1 = t_2$$

A **key** for a table is a set of attributes that **uniquely identify a row**

$\implies$ no two rows can have the same values for key attributes

Key constraints: special case of FDs $X \rightarrow Y$

where Y is the **whole set of attributes** of a relation

5/16

Inclusion dependencies (INDs)

Constraints of the form $R[X] \subseteq S[Y]$

where R, S are relations and X, Y are **sequences** of attributes

Semantics

R and S satisfy $R[X] \subseteq S[Y]$ if

for every $t_1 \in R$ there exists $t_2 \in S$ such that $\pi_X(t_1) = \pi_Y(t_2)$

Important: the projection must respect the attributes order

INDs are **referential constraints**: **link** the contents of one table
with the contents of another table

A **foreign key** constraint is the conjunction of two constraints:

- $R[X] \subseteq S[Y]$ (an IND)
- Y is key for S (a key constraint)

6/16

Examples of INDs

Employees		Departments	
Name	Dep	Name	Mgr
John	Finance	Finance	John
Mary	HR	HR	Mary
John	HR	Sales	Linda
Linda	Finance		
Susan	Sales		

Which of the following INDs would the above relations satisfy?

- $\text{Employees}[\text{Dep}] \subseteq \text{Departments}[\text{Name}]$ **Yes**
- $\text{Employees}[\text{Name}] \subseteq \text{Departments}[\text{Mgr}]$ **No**
- $\text{Departments}[\text{Mgr}] \subseteq \text{Employees}[\text{Name}]$ **Yes**
- $\text{Departments}[\text{Mgr}, \text{Name}] \subseteq \text{Employees}[\text{Name}, \text{Dep}]$ **No**

7/16

Basic SQL constraints

NOT NULL to disallow null values

UNIQUE to declare keys

PRIMARY KEY key + not null

FOREIGN KEY to reference attributes in other tables

NULL values are ignored when checking constraints except for **NOT NULL** and **PRIMARY KEY**

8/16

Not Null

Declaring an attribute as **NOT NULL** disallows **null values** for that attribute

```
CREATE TABLE Account (  
    accnum  VARCHAR(12) NOT NULL,  
    branch  VARCHAR(30),  
    custid  VARCHAR(10),  
    balance NUMERIC(14,2) DEFAULT 0  
);
```

The following insertion would fail:

```
INSERT INTO Account(branch , custid )  
VALUES ( 'London ' , 'cust1 ' );
```

9/16

Keys

```
CREATE TABLE Account (  
    accnum  VARCHAR(12) UNIQUE,  
    branch  VARCHAR(30),  
    custid  VARCHAR(10),  
    balance NUMERIC(14,2)  
);
```

The following insertion gives an error:

```
INSERT INTO Account VALUES  
(1, 'London',      'cust1', 100),  
(1, 'Edinburgh',  'cust3', 200);
```

The following insertion succeeds:

```
INSERT INTO Account VALUES  
(NULL, 'London',      'cust1', 100),  
(NULL, 'Edinburgh',  'cust3', 200);
```

10/16

Compound keys

Keys consisting of more than one attribute must be declared using a different syntax

```
CREATE TABLE Movies (  
    m_title      VARCHAR(30),  
    m_director   VARCHAR(30),  
    m_year       SMALLINT,  
    m_genre      VARCHAR(30),  
    UNIQUE (m_title,m_year)  
);
```

This declares the set {m_title,m_year} as a key for Movies

11/16

Primary Keys

Essentially **UNIQUE** + **NOT NULL**

```
CREATE TABLE Account (  
    accnum VARCHAR(12) PRIMARY KEY,  
    branch VARCHAR(30),  
    custid VARCHAR(10),  
    balance NUMERIC(14,2)  
);
```

same as

```
CREATE TABLE Account (  
    accnum VARCHAR(12) NOT NULL UNIQUE,  
    branch VARCHAR(30),  
    custid VARCHAR(10),  
    balance NUMERIC(14,2)  
);
```

12/16

Foreign keys in SQL (1)

```
CREATE TABLE Customer (  
    id VARCHAR(10) PRIMARY KEY  
    name VARCHAR(20),  
    city VARCHAR(30),  
    address VARCHAR(30)  
);
```

```
CREATE TABLE Account (  
    accnum VARCHAR(12),  
    branch VARCHAR(30),  
    custid VARCHAR(10) REFERENCES Customer(id),  
    balance NUMERIC(14,2)  
);
```

Every value for attribute custid in Account must appear among the values of the **key** id in Customer

13/16

Foreign keys in SQL (2)

General syntax (useful for declaring compound foreign keys)

```
CREATE TABLE <table1> (  
    <attr> <type>,  
    ...  
    <attr> <type>,  
    FOREIGN KEY (<list1>  
        REFERENCES <table2>(<list2>)  
);
```

where

- <list1> and <list2> are lists with the **same number** of attributes
- attributes in <list1> are from table <table1>
- attributes in <list2> are **unique** in <table2>

14/16

Referential integrity and database modifications (1)

Deletion can cause problems with foreign keys

<u>Customer</u>	<u>ID</u>	<u>Name</u>	<u>Account</u>	<u>Number</u>	<u>CustID</u>
	cust1	John		123456	cust1
	cust2	Mary		654321	cust2

where Account.CustID is a foreign key reference to Customer.ID

What happens if one deletes (cust1,John) from Customer?

Three approaches are supported in SQL:

1. Reject the deletion operation
2. Propagate it to Account by deleting also (123456,cust1)
3. "Don't know" approach: keep the tuple in Account, but set CustID value to **NULL**

15/16

Referential integrity and database modifications (2)

All three approaches are supported in SQL

```
CREATE TABLE <table1> (  
    <attr> <type>,  
    ...  
    FOREIGN KEY <list1> REFERENCES <table2> (<list2>)  
    <approach>  
)
```

where <approach> can be:

1. Empty: Reject deletions from <table2> causing the FK to be violated
(this is the default when <approach> is not specified)
2. **ON DELETE CASCADE**: Propagate the deletion to <name>
(tuples in <table1> that violate the FK will be deleted)
3. **ON DELETE SET NULL**: “Don’t know” approach
(the values of the attributes in <list1>, for tuples in <name>
that violate the FK, are set to **NULL**)

Introduction to Databases

(INFR10080)

(Entailment of Constraints)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Implication of constraints

A set Σ of constraints **implies** (or **entails**) a constraint ϕ if

every instance that satisfies Σ also satisfies ϕ

Notation: $\Sigma \models \phi$

Implication problem

Given Σ and ϕ , does Σ imply ϕ ?

Important because

- We never get the list of all constraints that hold in a database
- The given constraints may look fine, but imply some bad ones
- The given constraints may look bad, but imply only good ones

2/13

Axiomatization of constraints

Set of rules (**axioms**) to derive constraints

Sound every derived constraint is implied

Complete every implied constraint can be derived

Sound and **complete** axiomatization gives a procedure $\vdash$ such that

$$\Sigma \models \phi \quad \textbf{if and only if} \quad \Sigma \vdash \phi$$

3/13

Notation

Attributes are denoted by $A, B, C, \dots$

If A and B are attributes, AB denotes the set $\{A, B\}$

Sets of attributes are denoted by $X, Y, Z, \dots$

If X and Y are sets of attributes, XY denotes their union $X \cup Y$

If X is a set of attributes and A is an attribute,

XA denotes $X \cup \{A\}$

4/13

Armstrong's axioms

Sound and complete axiomatization for FDs

Essential axioms

Reflexivity: If $Y \subseteq X$, then $X \rightarrow Y$

Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z

Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Other axioms

Union: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

Decomposition: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

5/13

Closure of a set of FDs

Let Σ be a set of FDs

The **closure** Σ^+ of Σ is the set of all FDs implied by the FDs in Σ

Can be computed using Armstrong's axioms

Example

Closure of $\{A \rightarrow B, B \rightarrow C\}$ (blackboard)

6/13

Attribute closure

The **closure** $C_\Sigma(X)$ of a set X of attributes w.r.t. a set Σ of FDs is the set of attributes we can derive from X using the FDs in Σ (i.e., all the attributes A such that $\Sigma \vdash X \rightarrow A$)

Properties

- $X \subseteq C_\Sigma(X)$
- If $X \subseteq Y$, then $C_\Sigma(X) \subseteq C_\Sigma(Y)$
- $C_\Sigma(C_\Sigma(X)) = C_\Sigma(X)$

Solution to the implication problem:

$$\Sigma \models Y \rightarrow Z \quad \text{if and only if} \quad Z \subseteq C_\Sigma(Y)$$

7/13

Closure algorithm

Input: a set Σ of FDs, and a set X of attributes

Output: $C_{\Sigma}(X)$, the closure of X w.r.t. Σ

1. $unused := \Sigma$
2. $closure := X$
3. **while** ($(Y \rightarrow Z) \in unused$ and $Y \subseteq closure$)
 $closure := closure \cup Z$
 $unused := unused - \{Y \rightarrow Z\}$
4. **return** $closure$

Example

Closure of A w.r.t. $\{AB \rightarrow C, A \rightarrow B, CD \rightarrow A\}$ (blackboard)

8/13

Keys, candidate keys, and prime attributes

Let R be a relation with set of attributes U and FDs Σ

$X \subseteq U$ is a **key** for R if $\Sigma \models X \rightarrow U$

Equivalently, X is a key if $C_{\Sigma}(X) = U$ (why?)

Candidate keys

Keys X such that, for each $Y \subset X$, Y is not a key

Intuitively, keys with a **minimal** set of attributes

Prime attribute: an attribute of a candidate key

9/13

Attribute closure and candidate keys

Given a set Σ of FDs on attributes U ,
how do we compute all candidate keys?

1. $ck := \emptyset$
2. $G :=$ DAG of the powerset 2^U of U
 - ▶ Nodes are elements of 2^U (sets of attributes)
 - ▶ There is an edge from X to Y if $Y \subseteq X$ and $X - Y = \{A\}$
3. Repeat until G is empty:
 - Find a node X without children
 - if $C_\Sigma(X) = U$:
 - $ck := ck \cup \{X\}$
 - Delete X and all its ancestors from G
 - else:
 - Delete X from G

10/13

Implication of INDs

Given a set of INDs, what other INDs can we infer from it?

Axiomatization

Reflexivity: $R[X] \subseteq R[X]$

Transitivity: If $R[X] \subseteq S[Y]$ and $S[Y] \subseteq T[Z]$, then $R[X] \subseteq T[Z]$

Projection: If $R[X, Y] \subseteq S[W, Z]$ with $|X| = |W|$,
then $R[X] \subseteq S[W]$

Permutation: If $R[A_1, \dots, A_n] \subseteq S[B_1, \dots, B_n]$,
then $R[A_{i_1}, \dots, A_{i_n}] \subseteq S[B_{i_1}, \dots, B_{i_n}]$,
where $i_1, \dots, i_n$ is a permutation of $1, \dots, n$

Sound and complete derivation procedure for INDs

11/13

FDs and INDs together

Given a set Σ of FDs and an FD ϕ , we can decide whether $\Sigma \models \phi$

Given a set Γ of INDs and an IND ψ , we can decide whether $\Gamma \models \psi$

What about $\Sigma \cup \Gamma \models \phi$ or $\Sigma \cup \Gamma \models \psi$?

This problem is **undecidable**: no algorithm can solve it

What if we consider only **keys** and **foreign keys**?

The implication problem is still **undecidable**

Unary inclusion dependencies (UINDs)

INDs of the form $R[A] \subseteq S[B]$ where A, B are attributes

The implication problem for FDs and UINDs is **decidable in PTIME**

12/13

Further reading

Abiteboul, Vianu, Hull. **Foundations of Databases**. Addison-Wesley, 1995

Chapter 8 Functional Dependencies

Chapter 9 Inclusion Dependencies

- Algorithm for checking implication of INDs
- Proof that implication of INDs is PSPACE-complete
- Undecidability proof for implication of FDs+INDs
- Axiomatization for FDs+UINDs

13/13